



Análisis y Propuesta de Mejora del Sistema de Cancelaciones en Uber
mediante Penalizaciones Inteligentes

FACULTAD DE INGENIERIA Y NEGOCIOS

AUTORES:

Hinojosa De la cruz Hendrikx
Saavedra Vela, Justin Osmar
Saavedra Vela, Teddy Jesus

CURSO:

SOFTWARE I: PROCESO PERSONAL Y CALIDAD

DOCENTE:

ING. RONALD MIGUEL SERRANO HERNANDEZ

LIMA-PERÚ

2026

Índice del Proyecto

1. Resumen Ejecutivo.....	
2. Descripción del Caso Práctico.....	
3. Objetivos del Informe	
3.1. Objetivo General	
3.2. Objetivos Específicos	
4. Alcance de la Solución	
5. Actores y Requerimientos	
5.1. Sistema Actual	
5.2. Sistema Propuesto	
5.3. Cuadro Comparativo	
6. Modelado UML	
6.1. Diagramas de Casos de Uso (Actual y Propuesto)	
6.2. Diagramas de Clases (Actual y Propuesto)	
6.3. Diagramas de Actividades.....	
7. Ciclo de Vida y Proceso de Desarrollo.....	
8. Viabilidad del Proyecto	
8.1. Viabilidad Técnica y Tecnológica.....	
8.2. Viabilidad Económica	
8.3. Viabilidad del Cronograma.....	
9. Diseño del Sistema (HLD)	
9.1. Módulos del Sistema.....	
9.2. Diseño de Base de Datos.....	
10. Pruebas del Sistema (QA)	
10.1. Pruebas de Caja Negra.....	

10.1.1. Objetivos y Descripción	
10.1.2. Casos de Prueba (CN-01 al CN-08)	
10.1.3. Resultados y Conclusión	
10.2. Pruebas de Caja Blanca	
10.2.1. Objetivos y Descripción	
10.2.2. Funciones Analizadas	
10.2.3. Análisis de la Función Crítica (cancelar_conductor)	
10.2.4. Complejidad Ciclomática y Caminos Independientes.....	
11. Mantenimiento y Mejoras Futuras.....	
12. Conclusión del Proyecto.....	

RESUMEN EJECUTIVO

Para este informe analizamos el funcionamiento general de la plataforma Uber, que es usada en distintos países del mundo, la cual brinda el servicio de taxi conectando pasajeros con conductores mediante un sistema digital que gestiona los viajes en tiempo real. Debido a ello el aplicativo Uber se ha convertido en un aplicativo móvil muy importante en la vida diaria.

Sin embargo, también presenta algunos puntos bajos que están relacionadas con la gestión de viajes siendo una de las más relevantes la tasa de cancelación por parte de los conductores. Esta situación afecta directamente a los pasajeros generando pérdida de tiempo, incertidumbre y cobros injustificados en algunos casos. Además, se ha identificado conductas recurrentes donde se aceptan viajes y posteriormente se cancelan por motivos de conveniencia, distancia, tráfico entre otros.

Ante esta situación se propone el desarrollo de un sistema mejorado de gestión de cancelaciones basado en penalizaciones inteligentes según el desempeño del conductor. Dicha solución consistirá en implementar un modelo de sanciones progresivas donde, por cada cancelación injustificada por el conductor se le brindará un tiempo de suspensión temporal, tiempo en el cual no podrá recibir ningún otro viaje. Dicha suspensión aumentara gradualmente dependiendo del historial y numero de cancelaciones por parte del conductor.

De igual manera se implementará un sistema de puntuaciones parecidas al usado en las licencias de conducir, en el cual cada conductor contará con un puntaje inicial o de confiabilidad. Cada cancelación injustificada afectara negativamente este registro, disminuyendo progresivamente su puntuación según frecuencia de cancelación y gravedad de esta. En caso de que el puntaje llegue a cero se procederá a la suspensión permanente de la cuneta del conductor por incumplimiento de las políticas de servicio.

Al mismo modo también se añadirá la reasignación inmediata de viajes cancelados permitiendo que otro conductor cercano pueda tomar el viaje lo que reducirá el tiempo de espera del pasajero. Con ello se busca optimizar la plataforma y mejorar la satisfacción del usuario fomentando un entorno más confiable.

DESCRIPCION DEL CASO PRACTICO

Uber es una plataforma digital de transporte que permite la conexión entre pasajeros y conductores mediante su aplicativo. Uno de los principales problemas es la alta tasa de cancelaciones por parte de ciertos conductores, quienes aceptan solicitudes de viajes para posteriormente cancelarlas debido a distintos motivos como tráfico, distancia, zonas poco rentables entre otros.

Esta problemática ocasiona múltiples inconvenientes a los pasajeros, entre ellos largos tiempos de espera, retrasos incluso perdidas de confianza en el aplicativo. Los principales usuarios son:

- Pasajeros

- Conductores
- Plataforma

La necesidad principal consiste en mejorar el sistema de cancelaciones mediante mecanismos automáticos que permitan controlar el comportamiento reincidente, reducir las cancelaciones injustificadas y garantizar una reasignación rápida de los viajes cancelados.

OBJETIVOS DEL INFORME

Analizar la problemática de la tasa de cancelación en Uber y proponer un sistema inteligente de penalización y reasignación automática que permita mejorar la experiencia del usuario y optimizar la gestión de viajes.

Objetivos específicos

1. Identificar las principales causas y consecuencias de las cancelaciones dentro del sistema actual de Uber.
2. Diseñar un sistema de penalizaciones inteligentes basado en historial y puntuación progresiva similar a las licencias de conducir.
3. Proponer un mecanismo de reasignación inmediata de viajes cancelados para disminuir los tiempos de espera del pasajero.
4. Mejorar el control y monitoreo de conductores mediante indicadores de comportamiento y niveles de confiabilidad.⁴

ALCANCE DE SOLUCION

Este análisis se enfoca de manera principal en la mejora del servicio que se brinda por parte del sistema tomando como perspectiva principal al usuario(pasajero) y de manera secundaria al conductor.

- Registro y monitoreo de la tasa de cancelación de conductores.
- Implementación de penalizaciones progresivas según historial de cancelaciones.
- Sistema de puntuación tipo licencia de conducir.
- Reasignación automática inmediata de viajes cancelados.
- Generación de alertas y notificaciones en tiempo real.
- Clasificación de conductores según nivel de confiabilidad.
- Historial detallado de incidencias y cancelaciones.

ACTORES Y REQUERIMIENTOS

Sistema actual

- Pasajero
- Conductor
- Administrador del sistema Uber

Código	Tipo	Descripción	Prioridad
--------	------	-------------	-----------

RF-ACT-01	Funcional	Permitir al pasajero solicitar un viaje mediante la aplicación.	Alta
RF-ACT-02	Funcional	Asignar automáticamente un conductor disponible.	Alta
RF-ACT-03	Funcional	Permitir al conductor aceptar o cancelar viajes.	Alta
RF-ACT-04	Funcional	Mostrar información del viaje al pasajero y conductor.	Media
RF-ACT-05	Funcional	Registrar cancelaciones realizadas por pasajeros y conductores.	Alta
RF-ACT-06	Funcional	Calcular la tasa de cancelación del conductor.	Media
RF-ACT-07	Funcional	Permitir al administrador monitorear incidencias del sistema.	Media
RNF-ACT-01	No funcional	El sistema debe estar disponible las 24 horas del día.	Alta
RNF-ACT-02	No funcional	La aplicación debe responder rápidamente a solicitudes de viaje.	Alta
RNF-ACT-03	No funcional	El sistema debe proteger la información de usuarios y conductores.	Alta
RNF-ACT-04	No funcional	La interfaz debe ser sencilla e intuitiva.	Media

Sistema propuesto

- Pasajero
- Conductor
- Administrador del sistema
- Sistema inteligente de penalización

Código	Tipo	Descripción	Prioridad
RF-PROP-01	Funcional	Registrar automáticamente cada cancelación realizada por el conductor.	Alta
RF-PROP-02	Funcional	Aplicar suspensión temporal al conductor después de cada cancelación injustificada.	Alta
RF-PROP-03	Funcional	Incrementar progresivamente el tiempo de suspensión según reincidencia.	Alta
RF-PROP-04	Funcional	Implementar un sistema de puntuación similar a licencias de conducir.	Alta
RF-PROP-05	Funcional	Disminuir el puntaje del conductor por cancelaciones reiteradas.	Alta

RF-PROP-06	Funcional	Suspender permanentemente la cuenta cuando el puntaje llegue a cero.	Alta
RF-PROP-07	Funcional	Reasignar inmediatamente el viaje a otro conductor disponible.	Alta
RF-PROP-08	Funcional	Clasificar conductores según nivel de confiabilidad.	Media
RF-PROP-09	Funcional	Permitir al administrador visualizar historial y sanciones.	Media
RNF-PROP-01	No funcional	El sistema debe realizar reasignaciones en menos de 3 segundos.	Alta
RNF-PROP-02	No funcional	El sistema debe soportar múltiples usuarios simultáneamente.	Alta
RNF-PROP-03	No funcional	La información del historial debe almacenarse de forma segura.	Alta
RNF-PROP-04	No funcional	El sistema debe mantener disponibilidad continua del servicio.	Alta
RNF-PROP-05	No funcional	La interfaz debe mostrar claramente sanciones y puntuaciones.	Media

Cuadro comparativo

Aspecto	Sistema actual de Uber	Sistema propuesto
Gestión cancelaciones	de Las cancelaciones solo afectan parcialmente la tasa del conductor.	Las cancelaciones generan sanciones automáticas e inteligentes.
Penalización conductor	al Limitada y poco estricta.	Suspensión temporal progresiva después de cada cancelación injustificada.
Tiempo de sanción	Generalmente no existe bloqueo inmediato.	El tiempo de bloqueo aumenta gradualmente según reincidencia.
Sistema de puntuación	No existe un sistema visible similar a licencias.	Se implementa un sistema de puntos de confiabilidad.
Disminución de puntos	No se aplica reducción progresiva clara.	Cada cancelación reduce el puntaje del conductor.
Suspensión permanente	Solo ocurre en casos extremos evaluados manualmente.	El sistema bloquea automáticamente la cuenta al llegar a 0 puntos.
Reasignación de viajes	Puede tardar varios minutos o requerir nueva solicitud.	Reasignación inmediata automática a otro conductor cercano.
Experiencia del pasajero	Existen demoras, incertidumbre y cancelaciones frecuentes.	Menor tiempo de espera y mayor continuidad del servicio.

Control de reincidencia	Poco control sobre patrones repetitivos de cancelación.	Monitoreo constante y detección automática de conductores reincidentes.
Notificaciones	Básicas y limitadas.	Alertas automáticas y seguimiento en tiempo real.
Clasificación de conductores	Basada principalmente en calificaciones.	Clasificación basada en comportamiento, historial y confiabilidad.
Nivel de confiabilidad	Variable y difícil de controlar.	Mayor control y estabilidad del servicio.
Beneficio para la plataforma	Mantiene operación básica del servicio.	Optimiza la eficiencia y mejora la satisfacción del usuario.
Impacto en pasajeros	Pérdida de tiempo y mala experiencia.	Servicio más rápido, justo y confiable.
Automatización del sistema	Procesos limitados de control automático.	Sistema inteligente automatizado de sanciones y monitoreo.

MODELADO UML DEL CASO

Diagrama de casos de uso del sistema actual de Uber.

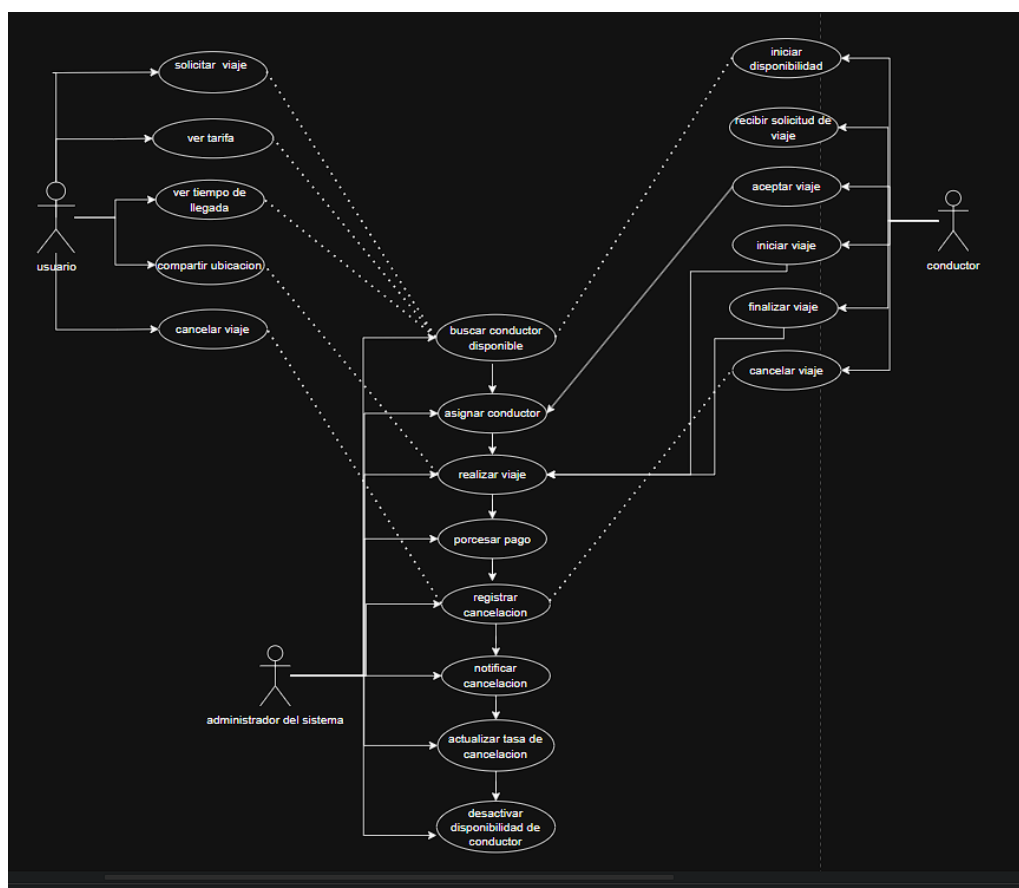


Diagrama de casos de uso del sistema propuesto.

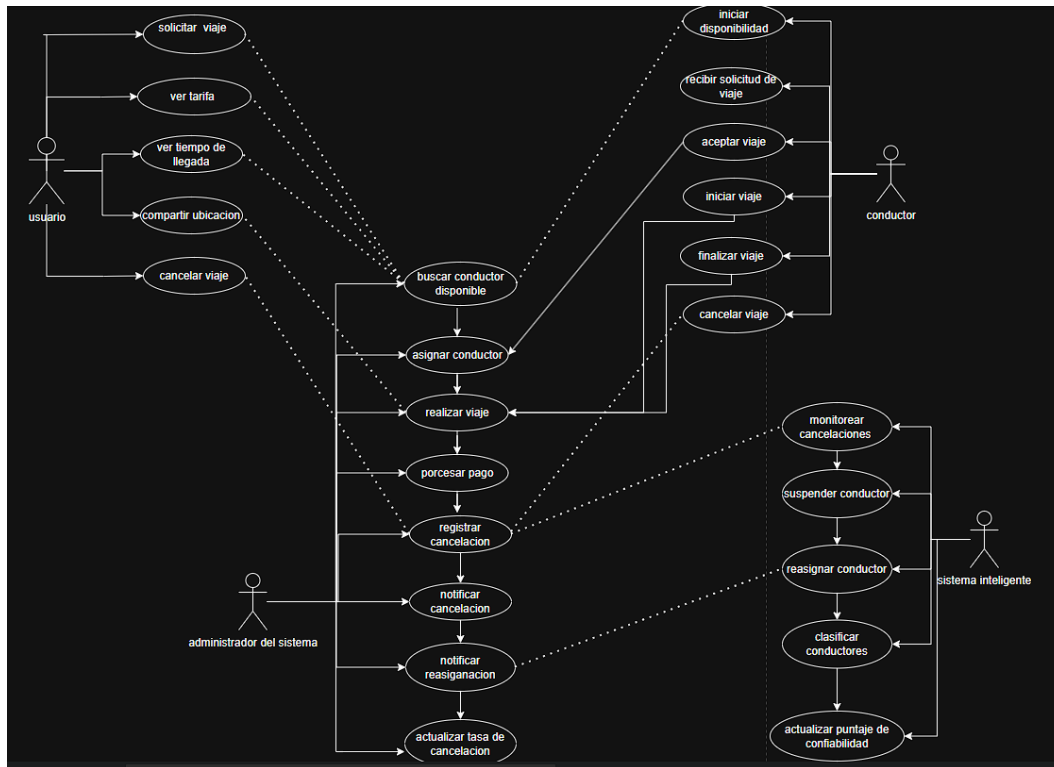


Diagrama de clases del sistema actual

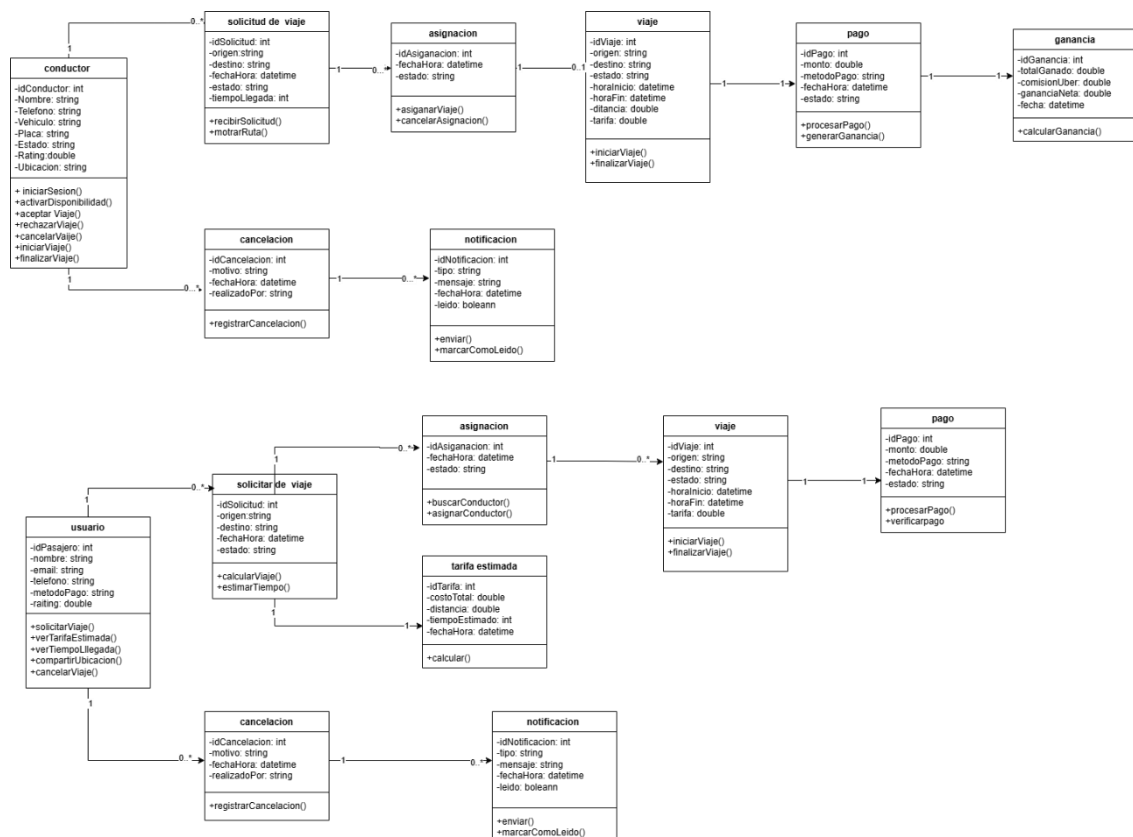


Diagrama de clases del sistema propuesto

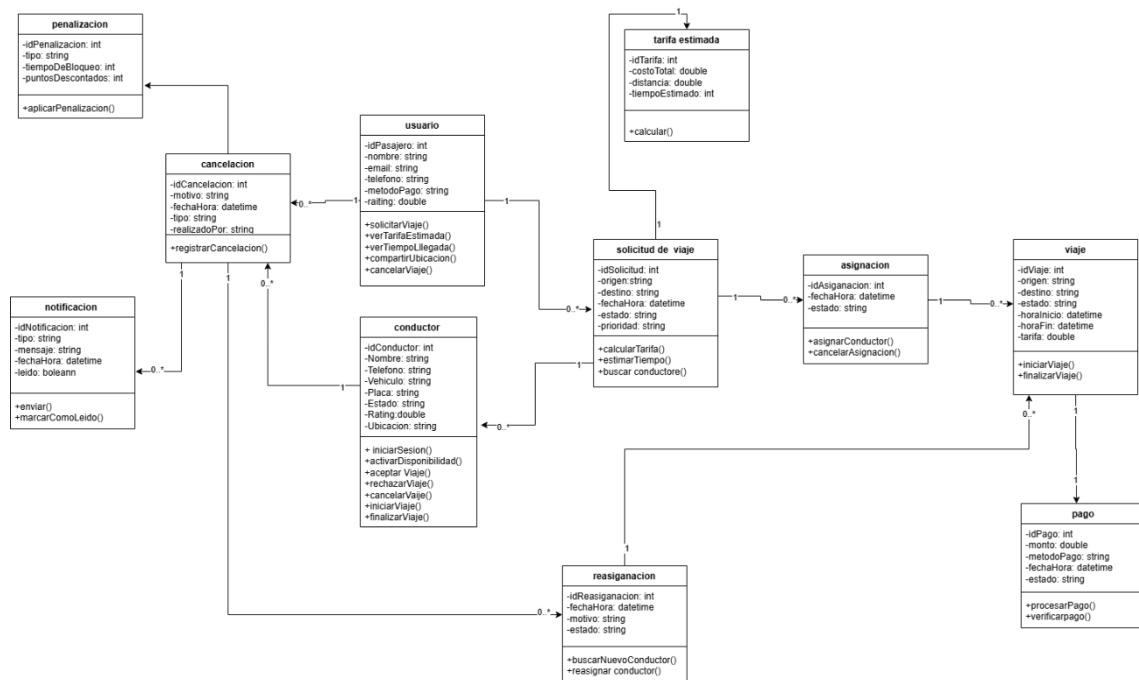


Diagrama de actividades del sistema actual

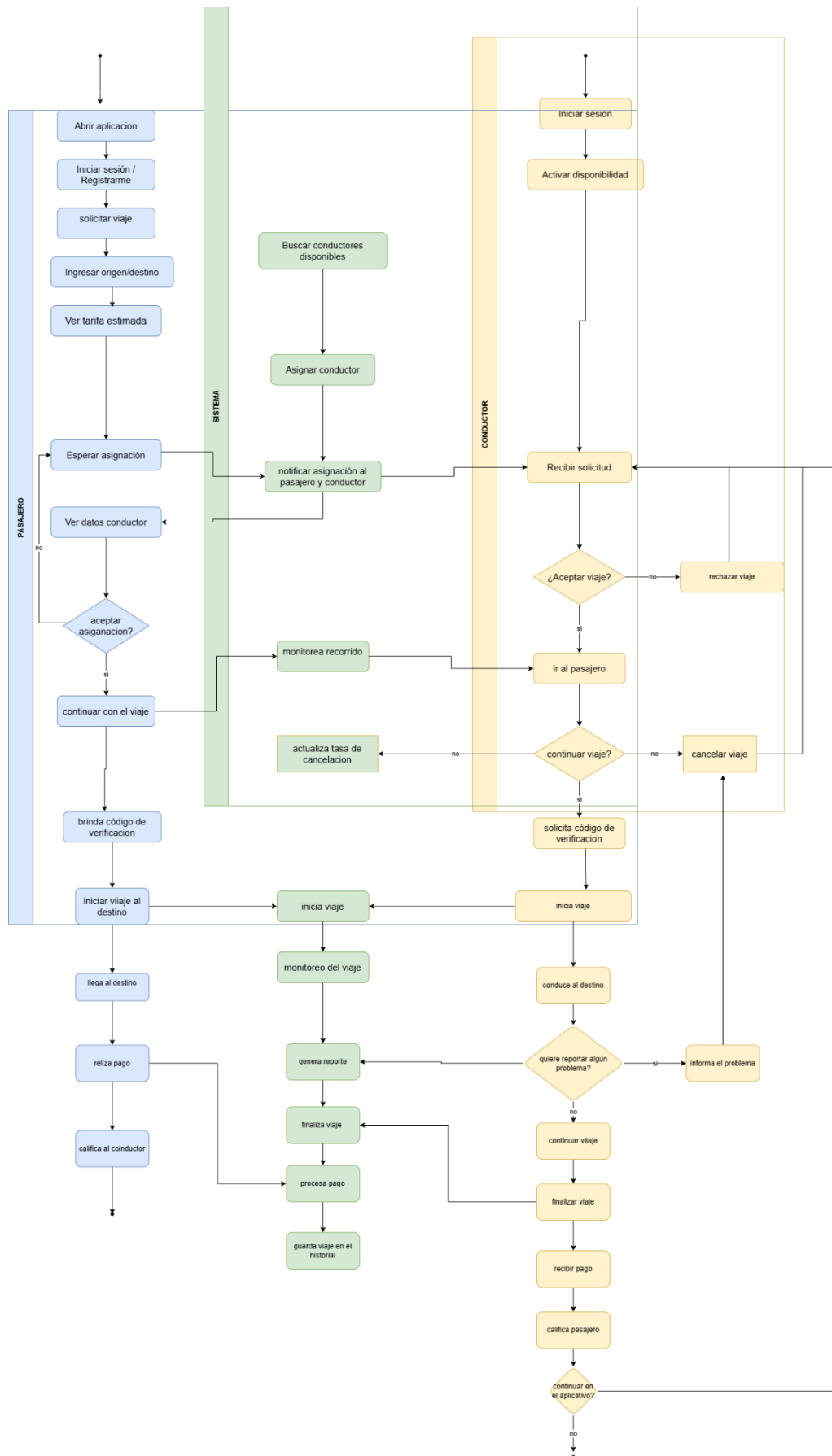
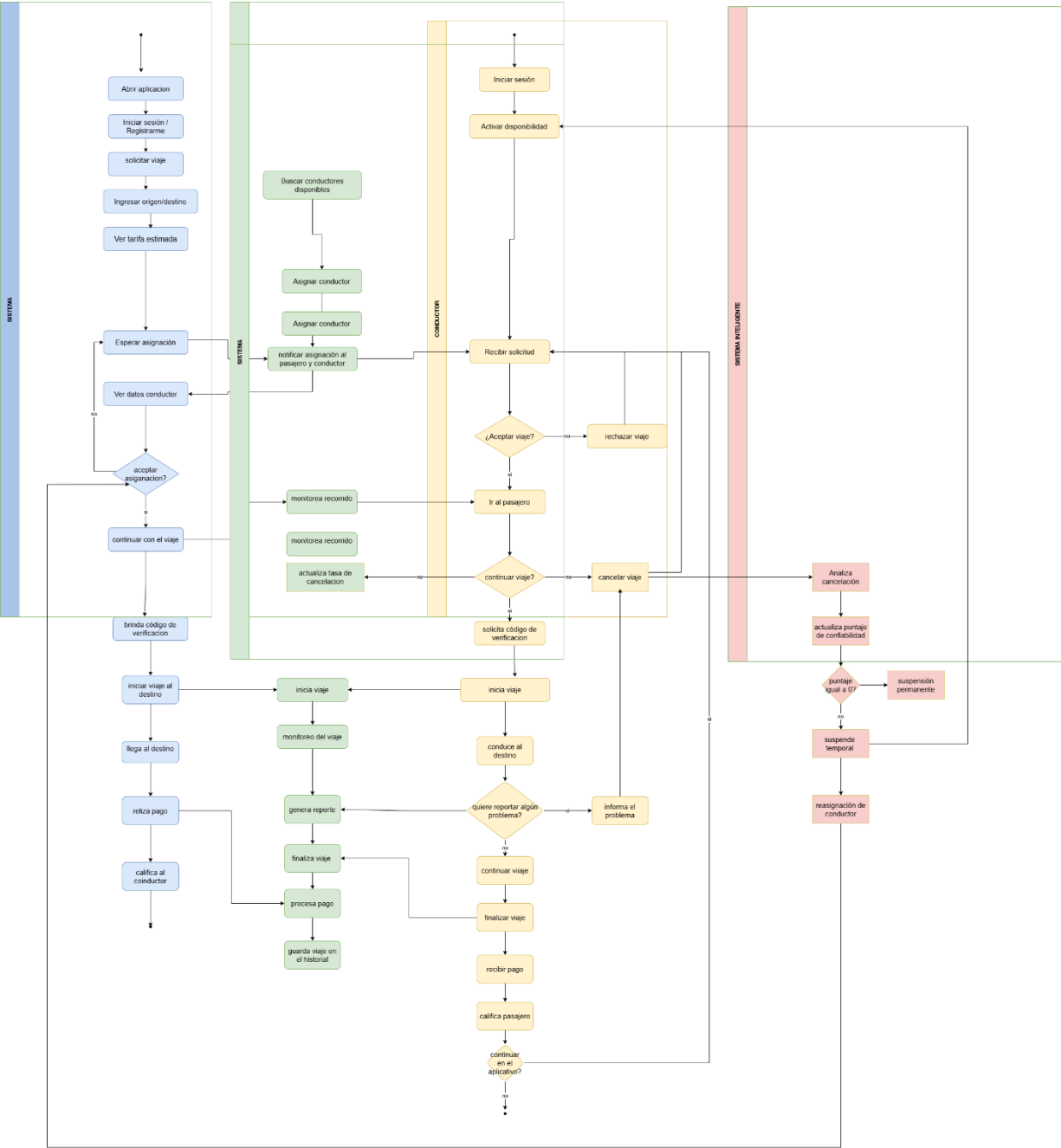
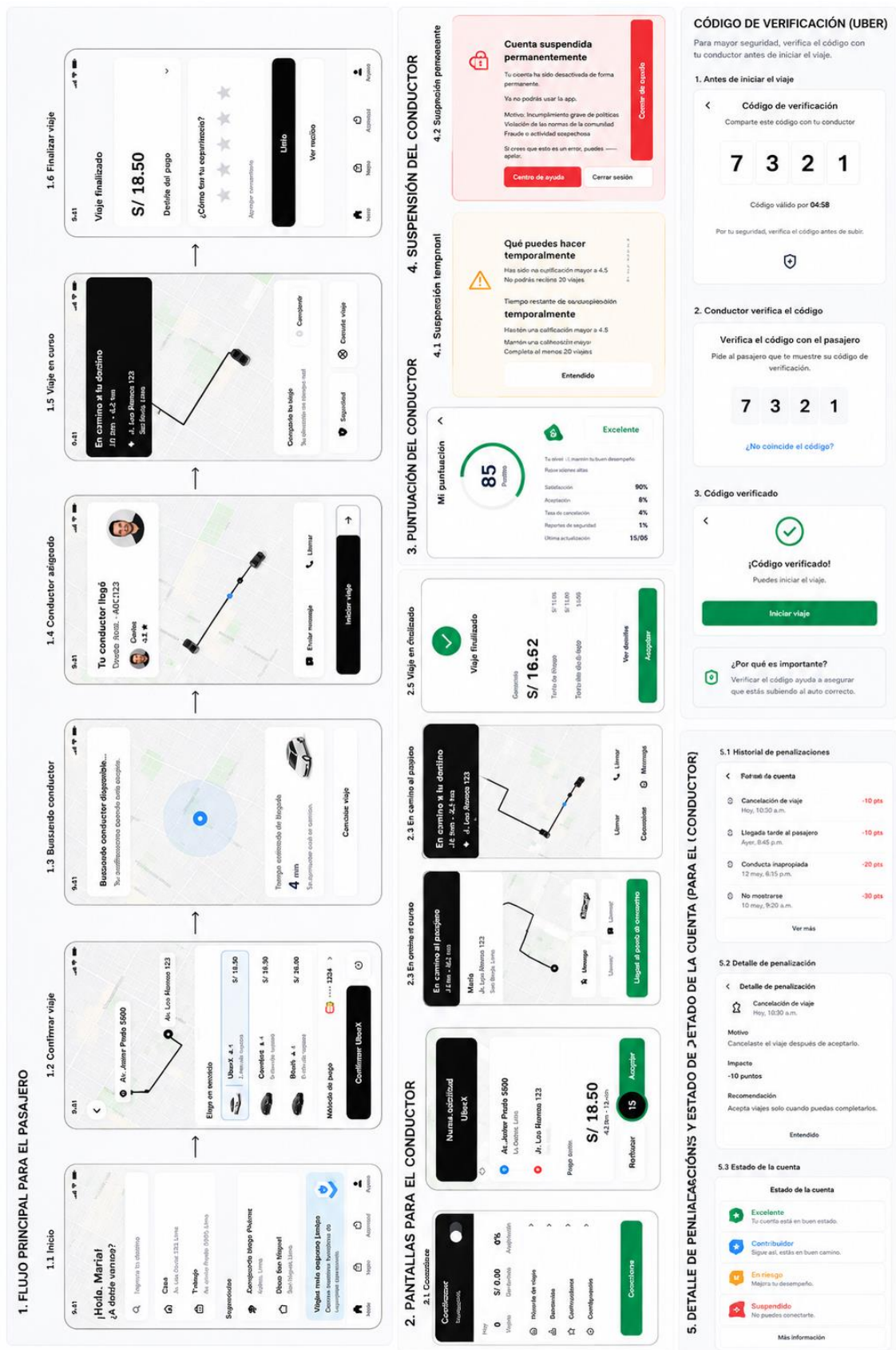


Diagrama de actividades del sistema propuesto





CICLO DE VIDA O PROCESO DE DESARROLLO

Modelo seleccionado	Justificación	Ventajas para el caso
Modelo Ágil	Se seleccionó el Modelo Ágil porque el sistema propuesto requiere cambios progresivos y mejoras continuas durante el desarrollo. Además, las funcionalidades planteadas como el sistema de penalizaciones, suspensión temporal, historial de incidencias, pueden implementarse por módulos o incrementos, permitiendo realizar pruebas constantes y obtener retroalimentación rápida. Este modelo también facilita adaptar el sistema según nuevas necesidades de los usuarios o cambios en las reglas de penalización y seguridad de la plataforma.	• Permite desarrollar funcionalidades por etapas. • Facilita realizar cambios rápidos durante el desarrollo. • Mejora la detección temprana de errores mediante pruebas continuas. • Permite entregar módulos funcionales en menor tiempo. • Favorece la adaptación de nuevas reglas de negocio y mejoras del sistema.

PLAN BASICO DE TRABAJO

N°	Actividad	Responsable	Fecha estimada	Producto esperado
1	Levantamiento de información del sistema actual	Analista sistemas	Semana 1	Documento de análisis actual
2	Identificación de problemas y requerimientos	Equipo de análisis	Semana 1	Lista de requerimientos funcionales y no funcionales
3	Diseño del sistema propuesto	Diseñador UML	Semana 2	Diagramas UML y propuesta funcional
4	Diseño de interfaz del conductor y pasajero	Diseñador UX/UI	Semana 3	Prototipo visual del sistema
5	Desarrollo del módulo de penalizaciones	Programador backend	Semana 4-5	Sistema automático de sanciones
6	Implementación de historial de incidencias	Programador backend	Semana 6	Registro de cancelaciones y penalizaciones
7	Desarrollo de suspensión temporal y progresiva permanente	Programador backend	Semana 7	Lógica de suspensión automática

8	Integración de validación código mediante	Programador móvil	Semana 8	Módulo de verificación de viaje
9	Pruebas funcionales y corrección de errores	Equipo QA	Semana 6	Sistema validado
10	Documentación final del proyecto	Equipo de desarrollo	Semana 7	Informe técnico final

CONTROL DE CALIDAD Y REVISION

Elemento a revisar	Criterio de calidad	Técnica de revisión	Evidencia
Módulo de penalizaciones	Correcto descuento de puntos	Prueba funcional	Registro de penalizaciones
Suspensión automática	Activación correcta según cancelaciones	Casos de prueba	Historial de suspensiones
Historial de incidencias	Visualización clara y completa	Revisión manual	Capturas del sistema
Sistema de puntuación	Actualización correcta del puntaje	Validación lógica	Tabla de puntuaciones
Verificación de viaje	Coincidencia correcta del código	Prueba de usuario	Confirmación de viaje seguro
Interfaz de usuario	Facilidad de uso y comprensión	Evaluación heurística	Retroalimentación de usuarios
Seguridad del sistema	Protección de datos y control de acceso	Revisión técnica	Informe de validación
Diagramas UML	Consistencia entre procesos	Revisión documental	Diagramas corregidos

PLAN BASICO DE PRUEBAS

Código	Funcionalidad	Tipo de prueba	Datos de entrada	Resultado esperado	Estado
PR-01	Registro de cancelación	Funcional	Cancelación realizada por conductor	Se registra la incidencia correctamente	Aprobado
PR-02	Descuento automático de puntos	Funcional	1 cancelación	Se descuentan 2 puntos automáticamente	Aprobado

PR-03	Suspensión temporal	Integración	Varias cancelaciones consecutivas	El sistema suspende al conductor por 5 minutos	Aprobado
PR-04	Historial de penalizaciones	Funcional	Incidencias registradas	Se muestran fechas, tiempo y puntos descontados	Aprobado
PR-05	Incremento progresivo de sanciones	Integración	4 cancelaciones consecutivas	El tiempo de suspensión aumenta progresivamente, hasta una suspensión permanente	Aprobado
PR-06	Actualización del puntaje	Funcional	Aplicación penalización	de El puntaje disminuye automáticamente	Aprobado
PR-07	Visualización de estado de cuenta	Funcional	Consulta conductor	del Se muestra el nivel de confiabilidad	Aprobado

PROPUESTA DE MEJORA

Nuestro sistema propuesto mejora significativamente la gestión de viajes por parte de la plataforma de Uber ya que permite incorporar sistemas de penalización automáticos, monitoreo del comportamiento de conducta por parte del conductor. A diferencia del sistema actual, donde no hay sanciones drásticas lo que afecta al usuario tanto en la pérdida de tiempo como en el aspecto económico, nuestra propuesta implementa un sistema detallado de las incidencias, suspensión temporal progresiva y disminución automática de puntos según la cantidad de cancelaciones realizada además de una suspensión definitiva cuando los puntos llegan a cero similar al usado en las licencias de conducir. Esta propuesta también mejora la experiencia del usuario ya que promueve conductas responsables por parte de los conductores y brinda mayor confiabilidad a la hora de brindar el servicio.

recopilación y análisis de requisitos

En una situación real, esta fase inicia con el análisis del funcionamiento actual de la plataforma Uber, enfocándose en el proceso de solicitud, aceptación y cancelación de viajes. Para ello, se recopila información sobre los inconvenientes que presentan los usuarios, principalmente las cancelaciones realizadas por algunos conductores después de haber aceptado un servicio. Esta información puede obtenerse mediante el análisis de reportes del sistema, reclamos de los pasajeros, indicadores de desempeño y las políticas actuales de la plataforma.

Posteriormente, se identifican los actores involucrados en el proceso, los cuales son el pasajero, el conductor, el administrador del sistema y el sistema

inteligente de penalización propuesto. A partir de sus necesidades se definen los requisitos que deberá cumplir la solución, considerando tanto los requerimientos funcionales, como el registro automático de cancelaciones, la aplicación de penalizaciones progresivas, la actualización del puntaje de confiabilidad y la reasignación inmediata del viaje, así como los requerimientos no funcionales relacionados con la disponibilidad del sistema, la seguridad de la información y la rapidez en la respuesta.

Durante esta etapa también se delimita el alcance del proyecto, estableciendo que la propuesta se enfocará en mejorar la gestión de las cancelaciones injustificadas por parte de los conductores, sin modificar el proceso general de solicitud de viajes ni otros servicios que ofrece la plataforma.

Como parte del análisis, se identifican los principales riesgos que podrían presentarse durante el desarrollo e implementación del sistema. Entre ellos se consideran posibles errores en la aplicación automática de las sanciones, inconvenientes en la reasignación de viajes y la necesidad de garantizar que las penalizaciones sean aplicadas únicamente cuando las cancelaciones sean realmente injustificadas. Estos riesgos permiten plantear medidas de control desde el inicio del proyecto.

Finalmente, toda la información recopilada es documentada y validada para asegurar que los requisitos representen correctamente las necesidades de la plataforma y de sus usuarios. Esta documentación servirá como base para el diseño, desarrollo y posterior implementación del sistema propuesto, reduciendo la posibilidad de realizar cambios importantes en etapas posteriores del proyecto.

Fase 2: Estudio de viabilidad

Una vez finalizado el análisis de requisitos, realizamos el estudio de viabilidad para determinar si la implementación del sistema de penalizaciones inteligentes y reasignación automática de viajes es conveniente para la plataforma Uber. Para ello, evaluamos la viabilidad económica, legal, operativa, técnica y del cronograma.

Viabilidad económica

Para este análisis planteamos un escenario referencial en el que el nuevo módulo sea implementado para la operación de Uber en Perú. El desarrollo sería realizado por un equipo pequeño durante dos meses.

Concepto	Costo estimado (USD)
Desarrollo del módulo de penalizaciones (2 desarrolladores Backend)	16,000
Desarrollo del módulo de reasignación automática	8,000
Desarrollo de interfaz y mejoras en la aplicación	4,000
Pruebas funcionales y control de calidad	3,000

Implementación y despliegue	2,500
Documentación y capacitación interna	1,500
Infraestructura y monitoreo inicial	5,000
Costo total estimado	40,000 USD

Considerando un tipo de cambio promedio de **S/ 3.70**, la inversión aproximada sería de:

$$40,000 \text{ USD} \times \text{S/ } 3.70 = \text{S/ } 148,000$$

Para comparar esta inversión con las pérdidas ocasionadas por las cancelaciones, planteamos un escenario basado en la operación diaria de la plataforma.

- Viajes diarios estimados: **120,000**
- Tasa de cancelación por parte de conductores: **5 %**
- Cancelaciones diarias: **6,000 viajes**
- Pérdida estimada por cada cancelación (compensaciones, soporte al cliente, promociones y pérdida de confianza): **S/ 4.50**

Esto representa una pérdida aproximada de:

- **Pérdida diaria:** $6,000 \times \text{S/ } 4.50 = \text{S/ } 27,000$
- **Pérdida mensual:** **S/ 810,000**
- **Pérdida anual:** **S/ 9,855,000**

Si con la implementación del sistema se logra reducir las cancelaciones injustificadas en un **40 %**, las pérdidas disminuirían aproximadamente en **S/ 3,942,000 al año**.

Al comparar la inversión inicial de **S/ 148,000** con el ahorro anual estimado de **S/ 3,942,000**, observamos que el costo de implementación representa menos del **4 %** del ahorro obtenido durante el primer año, por lo que el proyecto resulta económicamente rentable.

Viabilidad legal

Durante el desarrollo del sistema identificamos que la propuesta debe respetar tanto la legislación vigente como las políticas internas de la plataforma. En el caso del Perú, el tratamiento de la información de pasajeros y conductores deberá cumplir con la **Ley N.º 29733 - Ley de Protección de Datos Personales** y su reglamento, garantizando la confidencialidad y el uso adecuado de la información.

Asimismo, el sistema deberá respetar las **Políticas de Uso y Términos y Condiciones de Uber**, especialmente aquellas relacionadas con el comportamiento de los conductores, las cancelaciones de viajes, la suspensión de cuentas, el derecho de apelación y la transparencia en la aplicación de sanciones. También será necesario cumplir con los principios de seguridad de

la información para evitar accesos no autorizados a los historiales de incidencias y puntuaciones.

Viabilidad operativa

Desde el punto de vista operativo, determinamos que la propuesta no modifica la forma en que los pasajeros solicitan un viaje. El cambio se realiza principalmente en los procesos internos de la plataforma, donde el sistema registrará automáticamente las cancelaciones, calculará el puntaje del conductor, aplicará las sanciones correspondientes y reasignará el servicio a otro conductor cercano. Esto permite mejorar la experiencia del usuario sin alterar el funcionamiento habitual de la aplicación.

Viabilidad técnica

Para determinar la viabilidad técnica de nuestra propuesta, realizamos un análisis más detallado de la infraestructura tecnológica que utiliza Uber a nivel global. Al ser una plataforma que opera en múltiples países y procesa millones de viajes en tiempo real, su arquitectura está diseñada para ser altamente escalable, distribuida y tolerante a fallos.

En primer lugar, Uber no depende de un solo proveedor de infraestructura, sino que distribuye sus operaciones críticas entre tres grandes proveedores de nube pública: **Oracle Cloud Infrastructure (OCI)**, **Google Cloud Platform (GCP)** y **Amazon Web Services (AWS)**.

En el caso de **OCI**, Uber lo utiliza mediante acuerdos estratégicos para migrar cargas de trabajo críticas de producción. Este proveedor destaca por el uso de procesadores **ARM (Ampere)**, los cuales permiten reducir costos operativos y mejorar la eficiencia energética. Además, algunos servicios logísticos como **Uber Direct** se integran con **Oracle Retail** para facilitar las entregas de última milla.

Por otro lado, **Google Cloud Platform (GCP)** es uno de los pilares principales del sistema, ya que soporta el **Data Lake analítico de Uber**, el cual almacena más de **1.5 exabytes de información histórica**. En esta infraestructura se gestionan grandes volúmenes de datos y se utilizan balanceadores de carga ubicados en el “borde” de la red, lo que permite reducir la latencia y mejorar la velocidad de respuesta a nivel global.

Asimismo, **Amazon Web Services (AWS)** es utilizado principalmente para soportar las zonas de emparejamiento en tiempo real (Trip Serving Zones), que son responsables de conectar a pasajeros y conductores de manera inmediata. También se emplea para el entrenamiento de modelos de **Inteligencia Artificial (IA)**, los cuales ayudan a optimizar rutas, tiempos de llegada y predicción de demanda.

En cuanto al procesamiento de datos, Uber maneja un sistema de flujo en tiempo real basado en tecnologías de alto rendimiento. Entre los principales componentes se encuentran:

- **Apache Kafka**, que funciona como el sistema central de mensajería, transportando millones de eventos por segundo entre usuarios, conductores y servidores.
- **Apache Flink**, encargado de procesar y filtrar estos datos en tiempo real, permitiendo actualizar estados de viajes, ubicaciones y rutas de forma inmediata.
- **Hadoop**, utilizado para el almacenamiento y análisis de grandes volúmenes de datos históricos.

En el ecosistema de bases de datos, Uber utiliza diferentes tecnologías según el tipo de información:

- **Oracle Database**, para datos corporativos y financieros de alta seguridad.
- **DocStore (basado en MySQL)**, diseñado como motor transaccional distribuido para gestionar viajes activos en tiempo real.
- **Apache Cassandra**, utilizado para almacenar grandes volúmenes de datos de ubicación GPS sin afectar el rendimiento del sistema.
- **Redis**, empleado como base de datos en memoria para información temporal y de alta velocidad, como códigos de verificación o tarifas dinámicas.

Respecto al desarrollo del software, la plataforma está compuesta por miles de microservicios independientes desarrollados principalmente en:

- **Go (Golang)**, para servicios de alta concurrencia y geolocalización.
- **Java**, para lógica de negocio y procesamiento masivo.
- **Python**, utilizado en modelos de inteligencia artificial y análisis de datos.
- **Node.js**, para servicios web y mensajería en tiempo real.

Además, Uber ha desarrollado tecnologías propias de código abierto para resolver problemas específicos de su operación global. Entre ellas destacan:

- **H3 (Hexagonal Hierarchical Spatial Index)**, un sistema de indexación geoespacial que divide la Tierra en hexágonos para mejorar el análisis de oferta y demanda por zonas.
- **Cadence**, un sistema de orquestación de flujos de trabajo que garantiza que procesos complejos como cancelaciones, reembolsos y notificaciones se ejecuten correctamente, incluso si ocurren fallas en la red.

Toda esta infraestructura demuestra que Uber cuenta con un sistema altamente avanzado y distribuido, capaz de soportar grandes volúmenes de operaciones en tiempo real. Por lo tanto, la propuesta de implementación de un sistema de penalizaciones inteligentes puede integrarse como un nuevo microservicio dentro de esta arquitectura, sin necesidad de reemplazar componentes existentes.

Desde el punto de vista económico, esto reduce significativamente los costos del proyecto, ya que no se requiere inversión en servidores físicos ni en

infraestructura nueva. El impacto se limita principalmente al consumo adicional de recursos en la nube (procesamiento, almacenamiento y mensajería de eventos), lo cual representa un incremento mínimo en comparación con la capacidad total del sistema.

En conclusión, la infraestructura tecnológica de Uber es suficientemente robusta y flexible para soportar la implementación del sistema propuesto, lo que refuerza la viabilidad técnica y económica del proyecto.

Viabilidad del cronograma

Finalmente, estimamos que el proyecto puede desarrollarse en aproximadamente ocho semanas utilizando una metodología ágil. Este tiempo permite implementar cada módulo por etapas, realizar pruebas continuas y efectuar las correcciones necesarias antes de su puesta en producción, reduciendo los riesgos durante la implementación.

Después de analizar los aspectos económicos, legales, operativos, técnicos y de tiempo, concluimos que la propuesta es viable. Además de ser técnicamente posible, representa una inversión relativamente baja frente a las pérdidas económicas que generan las cancelaciones injustificadas, permitiendo mejorar la calidad del servicio y fortalecer la confianza de los usuarios en la plataforma.

Fase 3: Diseño

Una vez definidos los requisitos y comprobada la viabilidad del proyecto, procedimos con la fase de diseño del sistema. En esta etapa elaboramos la estructura general de la solución propuesta, tomando como base los requerimientos funcionales y no funcionales identificados anteriormente. El objetivo fue definir cómo estará organizado el sistema antes de iniciar su desarrollo.

Para nuestro proyecto diseñamos una arquitectura modular, de manera que cada componente cumpla una función específica y pueda trabajar de forma integrada con los demás. Esto facilitará el mantenimiento del sistema y permitirá incorporar futuras mejoras sin afectar el funcionamiento de toda la plataforma.

Diseño de Alto Nivel (HLD)

El sistema propuesto está conformado por los siguientes módulos:

1. Módulo de Gestión de Viajes

Este módulo administra todo el proceso relacionado con la solicitud de un viaje. Se encarga de recibir la solicitud del pasajero, buscar un conductor disponible, asignar el servicio y mantener el estado del viaje hasta su finalización o cancelación.

Funciones principales:

- Registrar solicitudes de viaje.
 - Asignar automáticamente un conductor disponible.
 - Actualizar el estado del viaje.
 - Enviar información del viaje al pasajero y conductor.
-

2. Módulo de Gestión de Cancelaciones

Este módulo registra todas las cancelaciones realizadas por los conductores y determina si corresponde aplicar una penalización según las reglas establecidas por el sistema.

Funciones principales:

- Registrar cada cancelación.
 - Verificar si la cancelación fue justificada o injustificada.
 - Enviar la información al sistema de penalizaciones.
 - Actualizar el historial del conductor.
-

3. Módulo de Penalizaciones Inteligentes

Es el módulo principal de la propuesta. Su función es evaluar el historial del conductor y aplicar automáticamente las sanciones correspondientes cuando exista una cancelación injustificada.

Funciones principales:

- Descontar puntos de confiabilidad.
 - Calcular el tiempo de suspensión temporal.
 - Incrementar las sanciones según la reincidencia.
 - Suspender permanentemente la cuenta cuando el puntaje llegue a cero.
-

4. Módulo de Reasignación Automática

Cuando un conductor cancela un viaje, este módulo busca inmediatamente otro conductor cercano para evitar que el pasajero tenga que realizar una nueva solicitud.

Funciones principales:

- Detectar viajes cancelados.
 - Buscar conductores disponibles en la zona.
 - Reasignar automáticamente el servicio.
 - Notificar la nueva asignación al pasajero.
-

5. Módulo de Monitoreo y Reportes

Este módulo permite al administrador consultar información relacionada con el comportamiento de los conductores y el funcionamiento del sistema.

Funciones principales:

- Visualizar historial de cancelaciones.
- Consultar sanciones aplicadas.
- Mostrar niveles de confiabilidad.
- Generar reportes estadísticos.

Relación entre los módulos

Los módulos trabajan de forma integrada durante todo el proceso del viaje. El Módulo de Gestión de Viajes controla el servicio desde su inicio. Si ocurre una cancelación, la información es enviada al Módulo de Gestión de Cancelaciones, el cual comunica el evento al Módulo de Penalizaciones Inteligentes para aplicar las sanciones correspondientes. Al mismo tiempo, el Módulo de Reasignación Automática busca otro conductor disponible para continuar con el servicio. Finalmente, toda la información queda registrada en el Módulo de Monitoreo y Reportes para futuras consultas por parte del administrador.

Diseño de la base de datos

Para el funcionamiento del sistema identificamos las principales tablas que almacenarán la información.

Tabla	Información almacenada	Llave principal
Usuarios	Datos de pasajeros y conductores	id_usuario
Conductores	Información del conductor, puntaje y estado de la cuenta	id_conductor
Viajes	Datos de cada viaje solicitado	id_viaje
Cancelaciones	Registro de todas las cancelaciones	id_cancelacion
Penalizaciones	Historial de sanciones aplicadas	id_penalizacion
Historial_Confiabilidad	Puntaje y nivel de confiabilidad del conductor	id_historial

Las relaciones entre estas tablas permiten consultar rápidamente el historial de un conductor, calcular sus sanciones y mantener un registro completo de todas las incidencias.

Arquitectura del sistema

La arquitectura propuesta sigue un modelo de tres capas, lo que facilita la organización y el mantenimiento del sistema:

- **Capa de presentación:** corresponde a la aplicación móvil utilizada por pasajeros y conductores, donde se realizan las solicitudes de viaje, se reciben notificaciones y se visualiza la información del servicio.
- **Capa de lógica de negocio:** procesa las reglas del sistema, administra las penalizaciones, calcula el puntaje de confiabilidad, controla las suspensiones y realiza la reasignación automática de viajes.
- **Capa de datos:** almacena toda la información relacionada con usuarios, viajes, cancelaciones, penalizaciones e historial de confiabilidad en una base de datos segura.

En esta fase también verificamos que el diseño propuesto fuera coherente con los diagramas UML elaborados anteriormente (casos de uso, clases y actividades), asegurando que cada módulo respondiera a los requisitos definidos durante el análisis. Con este diseño se establece una estructura clara que servirá como base para la etapa de desarrollo del sistema.

Fase 4: Codificación (detalle de implementación del sistema propuesto)

Una vez finalizada la fase de diseño, en esta etapa definimos cómo se implementaría el sistema propuesto a nivel de código, incluyendo los lenguajes, herramientas, nombres de variables, compiladores y depuradores que se utilizarían para el desarrollo del módulo de penalizaciones inteligentes y reasignación automática de viajes.

Lenguajes de programación utilizados

Para el desarrollo del sistema propuesto consideramos una arquitectura basada en microservicios, utilizando los siguientes lenguajes:

- **Java (Spring Boot):** para la lógica principal del sistema (penalizaciones, gestión de viajes y control de conductores).
- **Python:** para análisis de datos del comportamiento de conductores y cálculo de patrones de cancelación.
- **JavaScript (Node.js):** para servicios de notificaciones en tiempo real y comunicación con la aplicación móvil.

Entornos de desarrollo (IDE)

Los entornos de desarrollo utilizados serían:

- **IntelliJ IDEA:** principal entorno para el desarrollo en Java (backend del sistema).
- **Visual Studio Code:** utilizado para servicios en Node.js, Python y pruebas rápidas de APIs.
- **Android Studio (opcional):** para integración con la aplicación móvil del conductor y pasajero.

Compiladores e intérpretes

Dependiendo del lenguaje utilizado, se emplearían los siguientes:

- **JDK (Java Development Kit):** compilador para proyectos en Java.
- **Node.js Runtime:** intérprete para ejecutar servicios desarrollados en JavaScript.
- **Python Interpreter (CPython):** para ejecutar scripts de análisis y procesamiento de datos.
- **Maven:** para compilar, gestionar dependencias y construir el proyecto Java.

Depuradores (Debuggers)

Para detectar y corregir errores durante el desarrollo se utilizarían:

- **Debugger de IntelliJ IDEA:** para seguimiento de ejecución en Java.
- **Chrome DevTools:** para depuración de servicios web y APIs.
- **VS Code Debugger:** para Python y Node.js.
- **Postman Console:** para depuración de solicitudes API REST (errores en respuestas, tiempos de espera y validación de datos).

Herramientas de control y pruebas

- **Git y GitHub:** control de versiones del código fuente.
- **Postman:** pruebas de endpoints del sistema (cancelaciones, penalizaciones, reasignación).
- **JMeter (opcional):** pruebas de carga para simular múltiples usuarios solicitando viajes.
- **Docker:** para contenerización de microservicios y facilitar el despliegue.

Convenciones de nombres y variables del sistema

Para mantener el código organizado y fácil de entender, se definirían convenciones claras de nombres:

1. Variables principales del sistema

- `driverId` → identificador del conductor
- `passengerId` → identificador del pasajero
- `tripId` → identificador del viaje
- `cancellationId` → registro de cancelación
- `penaltyPoints` → puntos de confiabilidad del conductor
- `cancellationReason` → motivo de cancelación
- `isJustified` → indica si la cancelación es justificada o no
- `suspensionTime` → tiempo de suspensión del conductor
- `driverStatus` → estado del conductor (activo, suspendido, bloqueado)
- `reliabilityScore` → puntaje de confiabilidad

2. Funciones principales

- `registerCancellation()` → registra una cancelación de viaje

- `evaluateCancellation()` → analiza si la cancelación es justificada
- `applyPenalty()` → aplica reducción de puntos al conductor
- `calculateSuspensionTime()` → calcula el tiempo de suspensión progresiva
- `updateDriverScore()` → actualiza el puntaje de confiabilidad
- `blockDriverAccount()` → bloquea la cuenta si el puntaje llega a cero
- `reassignTrip()` → reasigna el viaje a otro conductor disponible
- `findNearestDriver()` → busca el conductor más cercano
- `notifyUser()` → envía notificaciones en tiempo real

3. Estructura básica de datos (ejemplo)

- Driver { id, name, rating, penaltyPoints, status }
- Trip { id, passengerId, driverId, status, location, time }
- Cancellation { id, tripId, driverId, reason, isJustified, date }
- Penalty { id, driverId, pointsDeducted, suspensionTime }

En esta fase de codificación se define cómo se construirá técnicamente el sistema propuesto, estableciendo herramientas, lenguajes y estructuras que permiten desarrollar un sistema organizado y escalable. Gracias a esta planificación, el sistema de penalizaciones inteligentes y reasignación automática puede implementarse de manera ordenada, asegurando claridad en el código, facilidad de mantenimiento y correcta integración entre módulos.

FASE DE PRUEBAS

Una vez finalizada la fase de codificación e integrado el sistema completo, procedimos a implementar el software en un entorno de pruebas con el objetivo de verificar su correcto funcionamiento. En esta etapa, el equipo de calidad (QA) es el encargado de evaluar cada una de las funcionalidades del sistema propuesto, asegurando que se cumplan los requisitos establecidos durante la fase de análisis.

El principal propósito de esta fase es comprobar que el sistema de penalizaciones inteligentes, la reasignación automática de viajes y el monitoreo de conductores funcionen de acuerdo con lo planificado. Para ello, se ejecutan diferentes tipos de pruebas como pruebas funcionales, de integración y de validación del sistema.

Durante el proceso de pruebas, el equipo de control de calidad revisa escenarios reales de uso, como la solicitud de un viaje, la cancelación por parte del conductor, la aplicación automática de sanciones y la reasignación inmediata del servicio a otro conductor disponible. De esta manera, se busca asegurar que el sistema responda correctamente en situaciones similares a las que ocurren en la vida real.

En caso de detectarse errores o fallas, estos son registrados y reportados al equipo de desarrollo. Posteriormente, los desarrolladores realizan las correcciones necesarias en el código y vuelven a enviar el sistema al entorno

de pruebas. Este ciclo se repite hasta que el sistema funcione de manera estable y cumpla con los requisitos definidos.

Entre los principales aspectos evaluados en esta fase se encuentran:

- Correcto registro de cancelaciones de viajes.
- Aplicación adecuada del sistema de penalizaciones.
- Disminución automática del puntaje de confiabilidad del conductor.
- Activación de suspensiones temporales y permanentes según corresponda.
- Funcionamiento de la reasignación automática de viajes en tiempo real.
- Precisión en la actualización del estado del viaje.
- Correcta visualización de la información en la interfaz de usuario.

Asimismo, se realizan pruebas de rendimiento para verificar que el sistema pueda soportar múltiples solicitudes de viajes al mismo tiempo sin afectar su funcionamiento. También se evalúa la seguridad del sistema, asegurando que la información de pasajeros y conductores esté protegida y que no existan accesos no autorizados.

Finalmente, la fase de pruebas concluye cuando el sistema cumple con todos los requisitos establecidos y funciona de manera estable, confiable y eficiente. Esto garantiza que la solución propuesta esté lista para pasar a la fase de implementación o despliegue, asegurando una mejora significativa en la gestión de cancelaciones dentro de la plataforma.

Fase 6: Instalación / Implementación

Una vez finalizada la fase de pruebas y habiendo verificado que el sistema funciona de manera estable y sin errores críticos, se procede con la fase de instalación o implementación. En esta etapa se realiza el despliegue del sistema de penalizaciones inteligentes y reasignación automática dentro del entorno real de operación.

Antes del lanzamiento final, se toma en cuenta la retroalimentación del gerente del proyecto y del equipo de calidad, con el fin de realizar los últimos ajustes necesarios. Esto permite asegurar que el sistema cumpla correctamente con los requisitos establecidos y que no existan problemas al momento de su integración con la plataforma.

La implementación del sistema se realiza de manera progresiva, es decir, no se activa todo el sistema de una sola vez, sino que se aplica por módulos. En primer lugar, se habilita el módulo de registro de cancelaciones y monitoreo de conductores, luego el sistema de penalizaciones progresivas y finalmente el módulo de reasignación automática de viajes en tiempo real.

Durante esta fase, el sistema es desplegado en los servidores de producción utilizando la infraestructura en la nube previamente establecida. Se configuran los servicios necesarios, las bases de datos y las conexiones con los

microservicios existentes para garantizar una integración correcta con el sistema principal de la plataforma.

Asimismo, se realiza un monitoreo constante del comportamiento del sistema en un entorno real, observando indicadores como tiempos de respuesta, número de cancelaciones registradas, aplicación de penalizaciones y eficiencia en la reasignación de viajes. Esto permite detectar posibles fallas menores que no hayan sido identificadas en la fase de pruebas.

En caso de presentarse inconvenientes, se realizan ajustes inmediatos por parte del equipo de desarrollo, asegurando la estabilidad del sistema sin afectar la experiencia del usuario final.

Finalmente, la fase de implementación concluye cuando el sistema se encuentra completamente operativo en el entorno real, funcionando de manera estable y cumpliendo con los objetivos planteados. Con ello, la plataforma logra mejorar la gestión de cancelaciones, reducir tiempos de espera y brindar un servicio más confiable para los usuarios.

Fase 7: Mantenimiento

Una vez implementado el sistema y puesto en funcionamiento para los usuarios, se inicia la fase de mantenimiento. En esta etapa se asegura que el sistema de penalizaciones inteligentes y reasignación automática continúe funcionando correctamente, y se realizan ajustes según el comportamiento real de la plataforma.

Durante esta fase se pueden realizar tres tipos de acciones principales: correcciones, actualizaciones y mejoras.

1. Corrección de errores

En el uso real del sistema pueden aparecer fallas que no fueron detectadas en las pruebas. Algunas correcciones que podrían ser necesarias en nuestro sistema son:

- Corrección de errores en el **registro de cancelaciones**, cuando no se almacena correctamente la información del viaje.
- Ajustes en la **reasignación automática**, si en algunos casos no se asigna un conductor cercano disponible.
- Corrección del cálculo de **puntos de confiabilidad**, cuando el sistema descuenta más o menos puntos de lo establecido.
- Solución de retrasos en la **actualización del estado del viaje** (por ejemplo, viaje cancelado que aún aparece como activo).
- Corrección de fallas en el envío de **notificaciones en tiempo real** a pasajeros y conductores.
- Ajustes en casos donde una cancelación injustificada sea clasificada incorrectamente como justificada.

2. Actualización del sistema (Upgrade)

Las actualizaciones se realizan para mantener el sistema compatible con nuevas tecnologías y políticas de la plataforma. En nuestro caso, algunas actualizaciones podrían ser:

- Actualización de las **reglas de penalización** según nuevas políticas de la empresa.
- Mejora de la compatibilidad con nuevas versiones de **Android e iOS**.
- Actualización de las librerías de backend (Spring Boot, Node.js, etc.) para mejorar seguridad y rendimiento.
- Optimización del sistema de bases de datos para manejar mayor volumen de viajes.
- Implementación de nuevas versiones del sistema de mensajería en tiempo real (por ejemplo, mejoras en Kafka o servicios de notificación).

3. Mejoras del sistema

Las mejoras se enfocan en agregar nuevas funcionalidades o hacer el sistema más eficiente. Para nuestro proyecto, algunas mejoras posibles son:

- Implementación de un sistema de **inteligencia artificial predictiva** para detectar conductores con alta probabilidad de cancelar antes de que acepten el viaje.
- Creación de un **sistema de reputación más avanzado**, que no solo considere cancelaciones, sino también tiempo de respuesta, aceptación de viajes y comportamiento general.
- Mejora del algoritmo de **reasignación automática**, considerando no solo la distancia, sino también el tráfico y el historial del conductor.
- Incorporación de un sistema de **alertas preventivas**, que advierta al conductor antes de realizar una cancelación.
- Desarrollo de un panel más detallado para administradores con **análisis estadístico en tiempo real**.
- Optimización del consumo de recursos para reducir costos en la infraestructura en la nube.

Finalmente, el objetivo del mantenimiento es garantizar que el sistema continúe funcionando de manera estable, eficiente y adaptándose a las necesidades reales de los usuarios. Esto permite que la solución propuesta siga mejorando con el tiempo y mantenga la calidad del servicio ofrecido por la plataforma.

CICLO DE PRUEBAS

Fase 1: Análisis de requisitos (STLC)

El análisis de requisitos dentro del Ciclo de Vida de Pruebas de Software (STLC) es la primera y una de las etapas más importantes del proceso de aseguramiento de calidad. En esta fase se revisan los requisitos del sistema desde una perspectiva orientada a pruebas, con el objetivo de identificar qué partes del software pueden ser evaluadas y cómo deben ser verificadas.

En nuestro caso, aplicamos esta fase al sistema propuesto de penalizaciones inteligentes y reasignación automática de viajes en la plataforma tipo Uber. El equipo de control de calidad analiza los requisitos funcionales y no funcionales definidos previamente, con el fin de asegurar que cada funcionalidad pueda ser validada correctamente antes de su implementación.

Durante esta etapa, el equipo de pruebas interactúa con los principales involucrados del proyecto, como el analista del sistema, el equipo de desarrollo y el responsable del proyecto, para comprender de manera clara el comportamiento esperado del sistema. Esto permite definir con precisión los escenarios de prueba relacionados con cancelaciones de viajes, aplicación de penalizaciones, actualización de puntajes y reasignación automática de conductores.

Entre las actividades principales realizadas en esta fase se incluyen:

- Identificación de las condiciones de prueba basadas en los requisitos del sistema.
- Definición de prioridades de prueba, dando mayor importancia a los módulos críticos como penalizaciones y reasignación de viajes.
- Elaboración de la Matriz de Trazabilidad de Requisitos (RTM), la cual permite relacionar cada requisito con sus respectivos casos de prueba.
- Identificación de necesidades de entorno de pruebas, como bases de datos de prueba, simulación de viajes y escenarios de cancelación.
- Evaluación de aspectos de seguridad, especialmente en el manejo de información de conductores y pasajeros.

Matriz de Trazabilidad de Requisitos (RTM)

La Matriz de Trazabilidad de Requisitos (RTM) permite relacionar cada requisito del sistema propuesto con sus respectivas pruebas, asegurando que todos los requerimientos sean validados correctamente durante el proceso de testing.

ID Requisito	Requisito	Tipo	Caso de prueba asociado	Resultado esperado
RF-PROP-01	Registrar cancelación del conductor	Funcional	PR-01	La cancelación se registra correctamente en el sistema
RF-PROP-02	Aplicar suspensión temporal al conductor	Funcional	PR-03	El conductor es suspendido según reglas establecidas
RF-PROP-03	Incrementar suspensión progresiva	Funcional	PR-05	El tiempo de suspensión aumenta según reincidencia

ID Requisito	Requisito	Tipo	Caso de prueba asociado	Resultado esperado
RF-PROP-04	Sistema de puntuación de confiabilidad	Funcional	PR-06	El puntaje se actualiza correctamente
RF-PROP-05	Disminuir puntaje por cancelaciones	Funcional	PR-06	El puntaje se reduce según reglas definidas
RF-PROP-06	Suspensión permanente por puntaje cero	Funcional	PR-05	El sistema bloquea la cuenta automáticamente
RF-PROP-07	Reasignación inmediata del viaje	Funcional	PR-05	El viaje es reasignado a otro conductor disponible
RNF-PROP-01	Reasignación en menos de 3 segundos	No funcional	PR-05	El sistema reasigna en tiempo real
RNF-PROP-02	Soporte de múltiples usuarios	No funcional	PR-07	El sistema soporta carga simultánea
RNF-PROP-03	Seguridad de información	No funcional	PR-04	Los datos se almacenan de forma segura

Viabilidad de pruebas

La viabilidad de pruebas en el sistema propuesto determina si es posible ejecutar correctamente todas las pruebas planificadas con los recursos disponibles, asegurando la calidad del software antes de su implementación final.

1. Viabilidad técnica

El sistema cuenta con una arquitectura basada en microservicios y tecnologías ampliamente utilizadas como Java, Node.js y Python, lo cual permite simular fácilmente los distintos escenarios de prueba. Además, el uso de herramientas como Postman, JUnit y bases de datos de prueba hace posible validar cada módulo de manera independiente.

Por lo tanto, técnicamente es viable ejecutar pruebas funcionales, de integración y de rendimiento sin limitaciones significativas.

2. Viabilidad de recursos

Se cuenta con herramientas gratuitas y ampliamente disponibles como Visual Studio Code, IntelliJ IDEA, GitHub y entornos de pruebas en la nube. Esto

permite realizar pruebas sin necesidad de una gran inversión adicional, utilizando entornos virtuales y bases de datos simuladas.

3. Viabilidad del entorno de pruebas

Es posible crear un entorno de pruebas similar al real mediante la simulación de usuarios, conductores y viajes. Esto permite replicar escenarios como cancelaciones, reasignaciones y aplicación de penalizaciones sin afectar datos reales.

4. Viabilidad del tiempo

El proceso de pruebas puede ejecutarse dentro del cronograma establecido del proyecto, ya que las pruebas están organizadas por módulos (penalizaciones, reasignación, monitoreo), lo que permite realizar validaciones progresivas sin retrasar el desarrollo general.

Se concluye que el proceso de pruebas es completamente viable, ya que el sistema cuenta con la tecnología, herramientas y metodología necesarias para ejecutar pruebas funcionales y no funcionales de manera eficiente. Esto garantiza que el sistema de penalizaciones inteligentes pueda ser validado correctamente antes de su implementación en producción.

Fase 2: Planificación de pruebas

El plan de pruebas del sistema de penalizaciones inteligentes y reasignación automática ha sido elaborado con el objetivo de garantizar la correcta validación de todas las funcionalidades del sistema antes de su implementación en producción.

Este documento establece los módulos a evaluar, los tipos de pruebas a realizar, los criterios de aceptación y los responsables del proceso de calidad. Asimismo, fue revisado y aprobado por el equipo de control de calidad y el responsable del proyecto, asegurando que cumple con los requisitos definidos en las fases anteriores del STLC.

El plan de pruebas incluye la validación de los siguientes componentes:

- Gestión de cancelaciones.
- Sistema de penalización progresiva.
- Cálculo de puntaje de confiabilidad.
- Reasignación automática de viajes.
- Notificaciones en tiempo real.

2. Estrategia de pruebas definida

La estrategia de pruebas adoptada para el sistema es de tipo **mixta**, combinando pruebas manuales y automatizadas.

- **Pruebas manuales:** Se utilizan para validar la interfaz de usuario, la experiencia del pasajero y conductor, y escenarios reales de cancelación.
- **Pruebas automatizadas:** Se aplican para validar procesos repetitivos como cálculo de penalizaciones, actualización de puntajes y respuesta de APIs.

El enfoque principal es garantizar la estabilidad del sistema en tiempo real, especialmente en la reasignación de viajes, que es una función crítica del sistema.

3. Estimación del esfuerzo y cronograma de ejecución

El esfuerzo de pruebas se estima en **5 semanas**, distribuidas de la siguiente manera:

- **Semana 1:** Preparación del entorno de pruebas y diseño de casos de prueba.
- **Semana 2:** Ejecución de pruebas unitarias por módulo.
- **Semana 3:** Pruebas de integración entre módulos.
- **Semana 4:** Pruebas de sistema completo (end-to-end).
- **Semana 5:** Corrección de errores, regresión y validación final.

El esfuerzo total estimado es de aproximadamente **200 horas de trabajo**, considerando actividades de diseño, ejecución, análisis de errores y revalidación.

4. Definición de herramientas y entorno de pruebas

Para la ejecución del proceso de pruebas se utilizan las siguientes herramientas:

- **Postman:** pruebas de APIs REST (cancelaciones, penalizaciones, reasignación).
- **JUnit:** pruebas unitarias en backend (Java).
- **Selenium:** pruebas de interfaz de usuario.
- **GitHub:** control de versiones y gestión de cambios.
- **MySQL Workbench:** validación de datos en base de datos.
- **Docker:** creación de entornos de pruebas aislados.
- **JMeter:** pruebas de carga y rendimiento.

El entorno de pruebas simula un sistema similar al de producción, utilizando bases de datos de prueba y usuarios ficticios (pasajeros y conductores).

5. Matriz inicial de riesgos de pruebas

Riesgo	Descripción	Impacto	Probabilidad	Mitigación
Fallo en reasignación automática	El sistema no asigna rápidamente un nuevo conductor	Alto	Media	Optimizar algoritmo de búsqueda de conductores
Error en cálculo de penalizaciones	Puntos mal descontados o aplicados incorrectamente	Alto	Media	Pruebas unitarias exhaustivas
Sobrecarga del sistema	Alta demanda simultánea de viajes	Alto	Media	Pruebas de rendimiento con JMeter
Fallas en notificaciones	El usuario no recibe alertas en tiempo real	Medio	Media	Uso de sistemas redundantes de mensajería
Inconsistencias en base de datos	Datos incorrectos en historial de conductores	Alto	Baja	Validaciones y backups constantes
Retraso en respuesta del sistema	Latencia en reasignación de viajes	Alto	Media	Optimización de consultas y caching

Estos entregables permiten estructurar el proceso de pruebas de manera clara y ordenada, asegurando que el sistema de penalizaciones inteligentes y reasignación automática sea evaluado de forma completa antes de su implementación final. Esto reduce riesgos, mejora la calidad del software y garantiza su correcto funcionamiento en un entorno real.

FASE 3

Casos de prueba / scripts de referencia y conjuntos de datos de prueba

Como parte de la fase de desarrollo de casos de prueba, se definieron los casos de prueba principales junto con sus scripts de referencia y los datos necesarios para su ejecución. Estos elementos permiten validar de manera estructurada el sistema de penalizaciones inteligentes y reasignación automática de viajes.

1. Casos de prueba y scripts de referencia

Los scripts de referencia representan la secuencia lógica de acciones que se deben ejecutar para validar cada funcionalidad del sistema.

ID	Caso de prueba	Script de referencia (acciones)	Resultado esperado
PR-01	Solicitud de viaje	Usuario solicita viaje → sistema busca conductor disponible	Conductor asignado correctamente

ID	Caso de prueba	Script de referencia (acciones)	Resultado esperado
PR-02	Asignación de conductor	Sistema asigna conductor automáticamente	Viaje con conductor activo
PR-03	Aceptación de viaje	Conductor acepta solicitud	Estado del viaje: "Aceptado"
PR-04	Cancelación por usuario	Usuario cancela viaje activo	Viaje cancelado correctamente
PR-05	Cancelación por conductor	Conductor cancela viaje → sistema registra evento	Cancelación registrada
PR-06	Aplicación de penalización	Sistema reduce puntaje del conductor	Puntaje disminuye según regla
PR-07	Suspensión temporal	Puntaje en nivel crítico → activar suspensión	Conductor suspendido temporalmente
PR-08	Suspensión permanente	Puntaje llega a 0 → bloqueo del conductor	Cuenta bloqueada definitivamente
PR-09	Reasignación de viaje	Conductor cancela → sistema reasigna viaje	Nuevo conductor asignado
PR-10	Registro de historial	Se ejecuta cualquier acción del sistema	Evento registrado en historial

1. Conjunto de datos de conductores

Se define una base de conductores simulados con distintos estados y niveles de confiabilidad:

- Conductor A: Puntaje 100 – Estado: Activo
- Conductor B: Puntaje 100 – Estado: Activo
- Conductor C: Puntaje 100 – Estado: Activo
- Conductor D: Puntaje 80 – Estado: Activo
- Conductor E: Puntaje 60 – Estado: Activo (riesgo de suspensión)
- Conductor F: Puntaje 40 – Estado: Suspendido temporal
- Conductor G: Puntaje 0 – Estado: Suspendido permanente

2. Conjunto de datos de pasajeros

Se simulan diferentes tipos de usuarios que generan solicitudes de viaje:

- Pasajero 1: Solicitud en zona urbana (alta demanda)
- Pasajero 2: Solicitud en zona periférica (media demanda)
- Pasajero 3: Solicitud en horario pico
- Pasajero 4: Solicitud en horario nocturno
- Pasajero 5: Solicitud en zona de baja cobertura

- Pasajero 6: Solicitud recurrente (usuario frecuente)

3. Conjunto de datos de viajes

Representa los distintos estados que puede tener un viaje dentro del sistema:

- Viaje V1: Estado pendiente
- Viaje V2: Estado aceptado
- Viaje V3: Cancelado por usuario
- Viaje V4: Cancelado por conductor
- Viaje V5: Reasignado automáticamente
- Viaje V6: En curso
- Viaje V7: Finalizado correctamente

4. Conjunto de datos de penalización

Define las reglas y valores utilizados para evaluar el comportamiento del conductor:

- Penalización por cancelación injustificada: -10 puntos
- Penalización por reincidencia: acumulativa progresiva
- Umbral de advertencia: 80 puntos
- Umbral de suspensión temporal: 70 puntos o menos
- Umbral de alto riesgo: 50 puntos o menos
- Suspensión permanente: 0 puntos

5. Conjunto de estados del sistema

Permite simular el comportamiento general de la plataforma:

- Estado del sistema: Activo
- Estado del sistema: En validación
- Estado del sistema: Reasignación en proceso
- Estado del sistema: Error controlado
- Estado del sistema: Mantenimiento

6. Conjunto de eventos del sistema

Se utilizan para validar el historial y trazabilidad:

- Evento: Solicitud de viaje creada
- Evento: Conductor asignado
- Evento: Viaje aceptado
- Evento: Cancelación registrada
- Evento: Penalización aplicada
- Evento: Suspensión temporal activada
- Evento: Suspensión permanente aplicada
- Evento: Viaje reasignado
- Evento: Viaje finalizado

Los casos de prueba y scripts de referencia, junto con los conjuntos de datos definidos, permiten ejecutar pruebas estructuradas y repetibles. Esto garantiza la correcta validación del sistema de penalizaciones y reasignación automática, asegurando coherencia entre los requisitos del sistema y los resultados esperados en cada escenario de prueba.

Fase 4: Configuración del entorno de prueba

La configuración del entorno de pruebas dentro del STLC es una fase técnica en la que se establecen las condiciones necesarias para ejecutar correctamente los casos de prueba del sistema. En esta etapa se prepara toda la infraestructura donde se validará el sistema de penalizaciones inteligentes y reasignación automática de viajes.

El objetivo principal es asegurar que el entorno simulado sea lo más similar posible al entorno real de operación, permitiendo así obtener resultados confiables durante la ejecución de las pruebas.

Importancia de la configuración del entorno

Una correcta configuración del entorno de pruebas permite evitar errores externos al sistema, como fallas de instalación, incompatibilidades o problemas de conectividad. Esto garantiza que los resultados obtenidos durante las pruebas sean atribuibles únicamente al sistema en evaluación.

Actividades realizadas en la configuración del entorno

1. Identificación de requisitos de hardware, software y red

Se definen los recursos necesarios para ejecutar el sistema de pruebas:

- Hardware:
 - Computadora con procesador de gama alta.
 - Memoria RAM mínima de 16 GB.
 - Almacenamiento disponible para bases de datos de prueba.
- Software:
 - Sistema operativo Windows o Linux
 - Entorno de desarrollo (IDE como VS Code o PyCharm)
 - Lenguajes de programación según el sistema (Python/Java según implementación)
 - Gestor de bases de datos (MySQL o PostgreSQL)
- Red:
 - Conexión a internet estable para simulación de servicios en tiempo real
 - Conectividad entre módulos del sistema

2. Instalación de componentes necesarios

Se procede a instalar y configurar los siguientes elementos:

- Sistema operativo compatible
- Entorno de desarrollo y herramientas de programación
- Base de datos de pruebas

- Servidor local para simulación del sistema
- Librerías necesarias para la ejecución del sistema de transporte simulado

3. Configuración de datos de prueba y conectividad

En esta etapa se integran los conjuntos de datos definidos previamente:

- Pasajeros simulados
- Conductores con distintos niveles de puntaje
- Historial de viajes y cancelaciones
- Reglas de penalización y suspensión

Además, se configura la conectividad entre los módulos del sistema para permitir el flujo completo de información (solicitud → asignación → cancelación → penalización → reasignación).

4. Ejecución de pruebas de humo (Smoke Testing)

Se realizan pruebas iniciales para verificar que el entorno funciona correctamente antes de ejecutar las pruebas completas. Estas pruebas incluyen:

- Verificación de que el sistema inicia correctamente
- Comprobación de conexión con la base de datos
- Validación de carga de usuarios simulados
- Ejecución básica de solicitud y asignación de viaje

Si estas pruebas son exitosas, el entorno se considera listo para la ejecución completa de los casos de prueba.

Lista de verificación de configuración del entorno

La lista de verificación permite confirmar que todos los componentes necesarios del entorno han sido instalados y configurados correctamente.

Checklist de configuración:

- Sistema operativo instalado y actualizado
- Entorno de desarrollo configurado (IDE operativo)
- Lenguaje de programación instalado correctamente
- Base de datos de pruebas creada y funcional
- Conexión entre sistema y base de datos verificada
- Librerías y dependencias del proyecto instaladas
- Servidor local configurado para ejecución del sistema
- Datos de prueba cargados correctamente
- Módulos del sistema integrados correctamente
- Red interna o conexión simulada operativa
- Acceso a herramientas de prueba habilitado

2. CHEKLIST de la prueba de humo (Smoke Testing)

Las pruebas de humo se realizaron con el objetivo de verificar que el sistema funciona de manera básica antes de ejecutar pruebas más complejas.

- El sistema inicia correctamente sin errores críticos
- La conexión con la base de datos se establece sin fallas
- El módulo de solicitud de viaje responde correctamente
- La asignación de conductor funciona de forma básica
- El sistema permite registrar cancelaciones
- El sistema ejecuta reglas básicas de penalización
- El historial de eventos registra acciones correctamente
- La interfaz responde de manera estable

3.CHEKLST DE Entorno de prueba completamente

Luego de ejecutar la verificación de configuración y las pruebas de humo, se concluye que el entorno de pruebas se encuentra completamente validado y listo para la ejecución de los casos de prueba del sistema.

El entorno permite:

- Simular usuarios (pasajeros y conductores)
- Ejecutar flujos completos de viaje
- Validar penalizaciones y suspensiones
- Probar la reasignación automática de conductores
- Registrar eventos en tiempo real
- Garantizar estabilidad durante la ejecución de pruebas

Fase 5: Ejecución de pruebas

La fase de ejecución de pruebas dentro del STLC es una de las etapas más importantes del proceso de aseguramiento de calidad, ya que en ella se valida directamente el comportamiento del sistema frente a los casos de prueba diseñados previamente. En esta fase se ejecutan las pruebas sobre el sistema de penalizaciones inteligentes y reasignación automática de viajes, utilizando el entorno de pruebas previamente configurado.

El objetivo principal es identificar defectos, verificar el cumplimiento de los requisitos y asegurar que el sistema funcione correctamente en escenarios reales simulados.

Actividades realizadas en la ejecución de pruebas

Durante esta fase se ejecutan los siguientes procesos:

- Ejecución de casos de prueba planificados para todos los módulos del sistema.
- Validación del flujo completo del sistema: solicitud, asignación, cancelación, penalización y reasignación.
- Registro de resultados de cada prueba con estado **Aprobado o Rechazado**.
- Documentación de defectos encontrados con su respectiva gravedad y prioridad.
- Reporte de errores al equipo de desarrollo para su corrección.
- Re-ejecución de pruebas luego de la corrección de errores (retesting).

- Ejecución de pruebas de regresión para asegurar que las correcciones no afecten funcionalidades existentes.

Ciclos de ejecución de pruebas

La ejecución de pruebas se realiza en varios ciclos para asegurar la calidad del sistema:

1. **Pruebas de cordura (Sanity Testing):**
Se verifica que las funcionalidades principales del sistema funcionen correctamente después de cambios menores.
2. **Pruebas de regresión:**
Se valida que las modificaciones no hayan afectado módulos ya probados, como asignación de conductores o penalizaciones.
3. **Re-testing (re-ejecución):**
Se vuelven a ejecutar los casos de prueba que anteriormente fallaron para confirmar que los errores fueron corregidos.

Registro de defectos

Durante la ejecución de pruebas, los defectos encontrados son registrados con la siguiente información:

- ID del defecto
- Descripción del error
- Módulo afectado
- Nivel de severidad (alto, medio, bajo)
- Prioridad de corrección
- Evidencia (capturas de pantalla o registros del sistema)
- Estado del defecto (abierto, en proceso, corregido, cerrado)

Métricas de evaluación

Para medir la calidad del sistema durante esta fase se consideran las siguientes métricas:

- Porcentaje de casos de prueba aprobados
- Número de defectos encontrados
- Densidad de defectos por módulo
- Tiempo de resolución de errores
- Tasa de re-ejecución exitosa

Entregables de la fase

- Matriz de trazabilidad de requisitos (RTM) actualizada con resultados de ejecución
- Reportes de resultados de pruebas (aprobados/rechazados)
- Registro de defectos del sistema
- Evidencias de pruebas ejecutadas
- Informes de regresión y re-testing

La fase de ejecución de pruebas permite validar de manera práctica el funcionamiento del sistema de penalizaciones inteligentes y reasignación automática de viajes. A través de múltiples ciclos de prueba, se asegura que el sistema cumpla con los requisitos establecidos, minimizando errores y garantizando la calidad del software antes de su implementación final.

Fase 6: Cierre del ciclo de pruebas

El cierre del ciclo de pruebas es la última fase del STLC y tiene como finalidad verificar que todas las pruebas planificadas hayan sido ejecutadas, que los defectos encontrados hayan sido corregidos o documentados y que el sistema cumpla con los requisitos definidos al inicio del proyecto.

En nuestro sistema propuesto de penalizaciones inteligentes y reasignación automática de viajes para Uber, esta fase nos permitió evaluar los resultados obtenidos durante las pruebas, identificar oportunidades de mejora y generar la documentación final del proceso de aseguramiento de la calidad.

Actividades realizadas

Durante esta fase se realizaron las siguientes actividades:

- Revisión de todos los casos de prueba ejecutados.
- Verificación del cumplimiento de los requisitos funcionales y no funcionales.
- Elaboración del informe de cierre de pruebas.
- Análisis de los defectos encontrados durante las pruebas.
- Identificación de los módulos con mayor cantidad de incidencias.
- Evaluación del porcentaje de pruebas aprobadas.
- Registro de las lecciones aprendidas para futuros desarrollos.
- Presentación de los resultados finales al equipo del proyecto.

Retrospectiva del proyecto

Al finalizar las pruebas se identificaron algunos aspectos que pueden mejorarse en futuras versiones del sistema:

- Optimizar el algoritmo de reasignación para disminuir aún más el tiempo de espera del pasajero.
- Mejorar la visualización del historial de sanciones para los conductores.
- Implementar notificaciones más detalladas cuando se aplique una penalización.
- Incorporar pruebas automáticas para reducir el tiempo de validación en futuras actualizaciones.
- Incrementar la cobertura de pruebas sobre escenarios excepcionales.

Métricas obtenidas

Para evaluar la calidad del sistema se recopilamos las siguientes métricas:

Métrica	Resultado
---------	-----------

Casos de prueba ejecutados	13
Casos de prueba aprobados	12
Casos de prueba rechazados	1
Porcentaje de aprobación	92.3 %
Defectos encontrados	3
Defectos críticos	0
Defectos de prioridad alta	1
Defectos de prioridad media	2
Defectos corregidos	3
Cobertura de requisitos funcionales	100 %
Cobertura de requisitos no funcionales	100 %

Análisis de resultados

Los resultados obtenidos muestran que el sistema cumple satisfactoriamente con los requisitos definidos para el proyecto. La mayoría de los casos de prueba fueron aprobados y los defectos encontrados fueron corregidos durante el proceso de validación.

Asimismo, las pruebas confirmaron que el sistema registra correctamente las cancelaciones, aplica las penalizaciones correspondientes, actualiza el puntaje de confiabilidad, realiza la suspensión temporal o permanente cuando corresponde y reasigna automáticamente los viajes cancelados a otro conductor disponible.

Plan básico de trabajo (Cronograma actualizado)

N.º	Actividad	Responsable	Semana	Producto esperado
1	Levantamiento de información del sistema actual de Uber	Analista de sistemas	Semana 1	Documento de análisis del sistema actual
2	Identificación de problemas y levantamiento de requisitos	Equipo de análisis	Semana 1	Requisitos funcionales y no funcionales
3	Estudio de viabilidad (económica, técnica, operativa, legal y de cronograma)	Analista de sistemas	Semana 2	Informe de viabilidad
4	Diseño del sistema propuesto y elaboración de diagramas UML	Diseñador UML	Semana 2	Diagramas UML completos

N.º	Actividad	Responsable	Semana	Producto esperado
5	Diseño de interfaces del pasajero, conductor y administrador	Diseñador UX/UI	Semana 3	Prototipos de interfaz
6	Desarrollo del módulo de gestión de viajes	Programador Backend	Semana 4	Módulo de solicitud y asignación de viajes
7	Desarrollo del módulo de penalizaciones inteligentes	Programador Backend	Semana 5	Sistema automático de sanciones
8	Desarrollo del sistema de puntuación y suspensión progresiva	Programador Backend	Semana 5	Módulo de puntuación y bloqueos
9	Desarrollo del módulo de reasignación automática de viajes	Programador Backend	Semana 6	Reasignación automática implementada
10	Implementación del historial de incidencias y monitoreo	Programador Backend	Semana 6	Historial de cancelaciones y sanciones
11	Integración de todos los módulos del sistema	Equipo de desarrollo	Semana 7	Sistema completamente integrado
12	Análisis de requisitos de pruebas (STLC)	Equipo QA	Semana 7	RTM y análisis de viabilidad de pruebas
13	Planificación de pruebas	Líder QA	Semana 8	Plan de pruebas y estrategia de pruebas
14	Desarrollo de casos de prueba y datos de prueba	Equipo QA	Semana 8	Casos de prueba, scripts y datos de prueba
15	Configuración del entorno de pruebas	QA y Administrador de sistemas	Semana 9	Entorno de pruebas configurado y validado
16	Ejecución de pruebas funcionales, integración y regresión	Equipo QA	Semana 9	Registro de resultados y reporte de defectos
17	Corrección de errores detectados	Equipo de desarrollo	Semana 10	Sistema corregido
18	Reejecución de pruebas (Retesting y Regresión)	Equipo QA	Semana 10	Validación final del sistema
19	Cierre del ciclo de pruebas y elaboración del informe de calidad	Líder QA	Semana 11	Informe final de pruebas y métricas

N.º	Actividad	Responsable	Semana	Producto esperado
20	Documentación final y presentación del proyecto	Equipo de desarrollo	Semana 12	Informe técnico completo y presentación final

Pruebas de caja negra

Las pruebas de caja negra verifican que el sistema funcione correctamente desde el punto de vista del usuario, sin analizar el código fuente. Se evalúan las entradas y salidas del sistema para comprobar que cumple con los requisitos funcionales.

Pruebas de Caja Negra

Código	Funcionalidad	Entrada	Resultado esperado	Resultado obtenido	Estado
CN-01	Solicitar viaje	El usuario solicita un viaje	El sistema asigna automáticamente un conductor disponible	Conductor asignado correctamente	Aprobado
CN-02	Aceptar viaje	El conductor acepta la solicitud	El estado cambia a "Aceptado"	Estado actualizado correctamente	Aprobado
CN-03	Cancelación por el usuario	El pasajero cancela el viaje	El sistema cambia el estado a "Cancelado por usuario"	Estado actualizado correctamente	Aprobado
CN-04	Cancelación por el conductor	El conductor cancela un viaje aceptado	El sistema registra la cancelación y aplica una sanción	Penalización aplicada correctamente	Aprobado
CN-05	Descuento de puntos	El conductor realiza una cancelación	Se descuentan 10 puntos del puntaje	Puntaje actualizado correctamente	Aprobado
CN-06	Suspensión temporal	El puntaje llega a 70 puntos	El conductor queda suspendido temporalmente durante 30 segundos	Suspensión aplicada correctamente	Aprobado
CN-07	Suspensión permanente	El puntaje llega a 0	La cuenta queda suspendida permanentemente	Suspensión aplicada correctamente	Aprobado
CN-08	Reasignación automática	El conductor cancela un viaje	El sistema asigna automáticamente otro conductor	Nuevo conductor asignado	Aprobado

CN-09	Reportar incidencia	El usuario registra una incidencia	El sistema almacena la incidencia y muestra un mensaje de confirmación	Incidencia registrada	Aprobado
CN-10	Historial	Se realiza cualquier operación	El historial muestra el registro correspondiente	Historial actualizado	Aprobado

2. Pruebas de Caja Blanca

Las pruebas de caja blanca analizan la lógica interna del programa, verificando que las funciones, decisiones, condiciones y rutas del código sean ejecutadas correctamente.

Pruebas de Caja Blanca

Código	Función evaluada	Condición del código	Resultado esperado	Estado
CB-01	hora()	Obtener la hora actual	Retorna la hora en formato HH:MM	Aprobado
CB-02	agregar historial()	Registrar un nuevo mensaje	El mensaje se agrega al historial del usuario y conductor	Aprobado
CB-03	actualizar()	Estado del conductor activo	Los botones permanecen habilitados	Aprobado
CB-04	actualizar()	Estado suspendido temporal	Los botones quedan deshabilitados	Aprobado
CB-05	actualizar()	Estado suspendido permanente	Los botones permanecen bloqueados	Aprobado
CB-06	solicitar_viaje()	Conductor suspendido permanentemente	El sistema muestra mensaje de error y no registra el viaje	Aprobado
CB-07	solicitar_viaje()	Conductor disponible	El viaje pasa al estado "Pendiente" y se asigna un conductor	Aprobado
CB-08	aceptar_viaje()	Existe un conductor asignado	El estado cambia a "Aceptado"	Aprobado
CB-09	cancelar_usuario()	Cancelación del pasajero	El estado cambia correctamente	Aprobado
CB-10	reportar_incidente()	Registro de incidencia	El historial se actualiza y aparece el mensaje de confirmación	Aprobado

CB-11	cancelar_conductor()	Puntaje mayor a 70	Se descuentan 10 puntos y continúa activo	Aprobado
CB-12	cancelar_conductor()	Puntaje igual o menor a 70	Se activa la suspensión temporal	Aprobado
CB-13	cancelar_conductor()	Puntaje igual a 0	Se activa la suspensión permanente	Aprobado
CB-14	contar_suspension()	Tiempo mayor a 0	El contador disminuye cada segundo	Aprobado
CB-15	contar_suspension()	Tiempo igual a 0	El conductor vuelve al estado Activo	Aprobado
CB-16	cancelar_conductor()	Reasignación de conductor	Se selecciona automáticamente el siguiente conductor disponible	Aprobado

EJECUCION DE PRUEBAS

Para verificar el correcto funcionamiento del sistema se realizaron **pruebas unitarias** utilizando la herramienta **Pytest**, la cual permite comprobar que cada función del programa responda de acuerdo con el resultado esperado.

Como primer paso, se instaló la herramienta mediante el comando:

```
pip install pytest
```

Una vez instalada, se ejecutaron todas las pruebas del proyecto con el siguiente comando:

```
pytest
```

Al ejecutar este comando, el sistema identificó automáticamente el archivo **test_uber_app.py**, en el cual se encontraban definidas las pruebas. Como resultado, se ejecutaron siete pruebas y todas fueron aprobadas correctamente (**7 passed**), lo que indica que las funciones evaluadas se comportaron de acuerdo con lo esperado.

Las pruebas realizadas fueron las siguientes:

- **Solicitud de viaje:** se verificó que al solicitar un viaje el estado cambiara a **Pendiente**.
- **Aceptación de viaje:** se comprobó que al aceptar un viaje el estado cambiara a **Aceptado**.
- **Cancelación del viaje por el usuario:** se verificó que el sistema cambiara el estado a **Cancelado por usuario**.
- **Descuento de puntaje al conductor:** se comprobó que al cancelar un viaje el conductor perdiera los puntos establecidos.

- **Suspensión temporal del conductor:** se verificó que cuando el puntaje descendiera al límite definido, el conductor quedara suspendido temporalmente.
- **Suspensión permanente del conductor:** se comprobó que, al disminuir aún más el puntaje, el sistema aplicara una suspensión permanente.
- **Validación de la función de hora:** se verificó que la función encargada de mostrar la hora generara el formato esperado.

Posteriormente, para conocer qué porcentaje del código fue evaluado durante las pruebas, se instaló la herramienta **Coverage** mediante el comando:

```
pip install coverage
```

Después se ejecutaron nuevamente las pruebas utilizando Coverage con el siguiente comando:

```
coverage run -m pytest
```

Al finalizar la ejecución, se generó el reporte de cobertura mediante el comando:

```
coverage report
```

El resultado obtenido mostró una **cobertura del 97 %**, lo que significa que prácticamente todo el código del sistema fue ejecutado durante las pruebas, quedando únicamente dos líneas sin evaluar. Este resultado demuestra que las pruebas realizadas cubren la mayor parte de la lógica implementada en la aplicación.

Finalmente, se generó un reporte visual utilizando el comando:

```
coverage html
```

Este comando creó una carpeta llamada **htmlcov**, dentro de la cual se encuentra el archivo **index.html**. Al abrir dicho archivo en el navegador fue posible visualizar el porcentaje de cobertura y observar qué partes del código fueron ejecutadas durante las pruebas, confirmando una cobertura total del **97 %**.

Instalación de **pytest** y **coverage.py**. para pruebas de caja blanca

```

PS C:\Users\USUARIO\OneDrive\Escritorio\sistema uber> pip install pytest
Collecting pytest
  Downloading pytest-9.1.1-py3-none-any.whl.metadata (7.6 kB)
Collecting colorama>=0.4 (from pytest)
  Downloading colorama-0.4.6-py2.py3-none-any.whl.metadata (17 kB)
Collecting iniconfig>=1.0.1 (from pytest)
  Downloading iniconfig-2.3.0-py3-none-any.whl.metadata (2.5 kB)
Requirement already satisfied: packaging>=22 in c:\users\usuario\AppData\Local\Programs\Python\Python313\lib\site-packages (from pytest) (25.0)
Collecting pluggy<2,>=1.5 (from pytest)
  Downloading pluggy-1.6.0-py3-none-any.whl.metadata (4.8 kB)
Collecting pygments>=2.7.2 (from pytest)
  Downloading pygments-2.20.0-py3-none-any.whl.metadata (2.5 kB)
Downloading pytest-9.1.1-py3-none-any.whl (386 kB)
Downloading colorama-0.4.6-py2.py3-none-any.whl (25 kB)
Downloading iniconfig-2.3.0-py3-none-any.whl (7.5 kB)
Downloading pluggy-1.6.0-py3-none-any.whl (20 kB)
Downloading pygments-2.20.0-py3-none-any.whl (1.2 MB)
1.2/1.2 MB 9.7 MB/s eta 0:00:00
Installing collected packages: pygments, pluggy, iniconfig, colorama, pytest
Successfully installed colorama-0.4.6 iniconfig-2.3.0 pluggy-1.6.0 pygments-2.20.0 pytest-9.1.1

[notice] A new release of pip is available: 25.0.1 -> 26.1.2
[notice] To update, run: python.exe -m pip install --upgrade pip
PS C:\Users\USUARIO\OneDrive\Escritorio\sistema uber> pip install coverage
Collecting coverage
  Downloading coverage-7.15.0-cp313-cp313-win_amd64.whl.metadata (8.8 kB)
Downloading coverage-7.15.0-cp313-cp313-win_amd64.whl (223 kB)
Installing collected packages: coverage
Successfully installed coverage-7.15.0

[notice] A new release of pip is available: 25.0.1 -> 26.1.2
[notice] To update, run: python.exe -m pip install --upgrade pip
PS C:\Users\USUARIO\OneDrive\Escritorio\sistema uber>

```

```

PS C:\Users\USUARIO\OneDrive\Escritorio\sistema uber> pytest
===== test session starts =====
platform win32 -- Python 3.13.3, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\USUARIO\OneDrive\Escritorio\sistema uber
collected 7 items

test_uber_app.py .....

===== 7 passed in 0.23s =====
PS C:\Users\USUARIO\OneDrive\Escritorio\sistema uber>

```

```

PS C:\Users\USUARIO\OneDrive\Escritorio\sistema uber> coverage run -m pytest
>> coverage report
===== test session starts =====
platform win32 -- Python 3.13.3, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\USUARIO\OneDrive\Escritorio\sistema uber
collected 7 items

test_uber_app.py .....

===== 7 passed in 0.21s =====

Name                Stmts   Miss  Cover
-----
test_uber_app.py      30      0   100%
uber_app.py           44      2    95%
TOTAL                  74      2    97%
PS C:\Users\USUARIO\OneDrive\Escritorio\sistema uber>

```

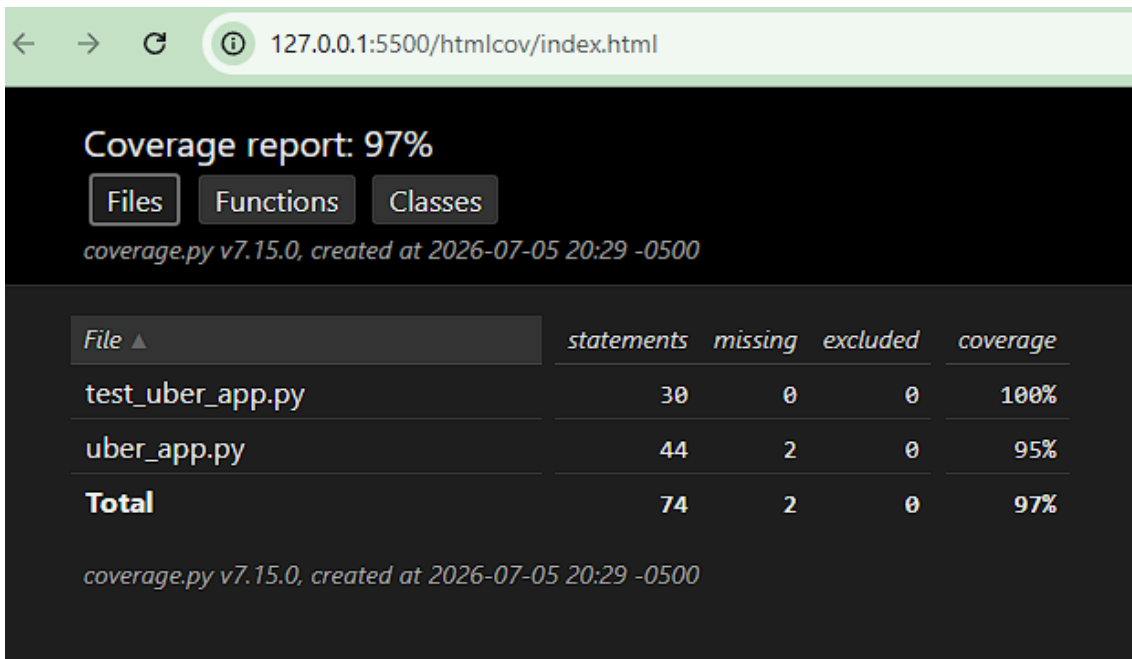


```

PS C:\Users\USUARIO\OneDrive\Escritorio\sistema uber> pytest -v
===== test session starts =====
platform win32 -- Python 3.13.3, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\USUARIO\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\USUARIO\OneDrive\Escritorio\sistema uber
plugins: cov-7.1.0
collected 7 items

test_uber_app.py::test_hora PASSED [ 14%]
test_uber_app.py::test_solicitar_viaje PASSED [ 28%]
test_uber_app.py::test_aceptar_viaje PASSED [ 42%]
test_uber_app.py::test_cancelar_usuario PASSED [ 57%]
test_uber_app.py::test_cancelar_conductor_descuento PASSED [ 71%]
test_uber_app.py::test_suspension_temporal PASSED [ 85%]
test_uber_app.py::test_suspension_permanente PASSED [100%]

```



Prueba unitaria	Resultado esperado	Estado
Solicitar viaje	Estado = Pendiente	Aprobado
Aceptar viaje	Estado = Aceptado	Aprobado
Cancelar viaje	Estado = Cancelado por usuario	Aprobado
Descuento de puntos	Puntaje disminuye en 10	Aprobado
Suspensión temporal	Estado = Suspendido temporal	Aprobado
Suspensión permanente	Estado = Suspendido permanente	Aprobado
Reasignación de conductor	Nuevo conductor asignado	Aprobado

REPOSITORIO

<https://github.com/flopy132017-hash/sistema-uber>

CAJA NEGRA

1. Objetivo

El objetivo de las pruebas de Caja Negra es verificar que el sistema funcione correctamente de acuerdo con los requisitos establecidos, evaluando únicamente las entradas y salidas del programa sin analizar la estructura interna del código fuente.

2. Descripción de la prueba

Las pruebas de Caja Negra se realizaron simulando las acciones que puede ejecutar un usuario dentro del sistema. Para ello, se verificó el comportamiento de las funciones principales relacionadas con la solicitud de viajes, aceptación, cancelación, reporte de incidencias y aplicación de sanciones, comprobando que las respuestas obtenidas fueran las esperadas.

3. Casos de prueba

4. Resultados obtenidos

Durante la ejecución de las pruebas se verificó que todas las funcionalidades

Caso	Acción	Entrada	Resultado esperado
CN-01	Solicitar viaje	Click en "Solicitar viaje"	Se asigna un conductor y el estado cambia a "Pendiente"
CN-02	Aceptar viaje	Click en "Aceptar Viaje"	El estado cambia a "Aceptado"
CN-03	Cancelar Viaje (Usuario)	Click en "Cancelar Viaje"	Estado: "Cancelado por usuario"
CN-04	Reportar incidencia	Click en "Reportar Incidencia"	Se muestra el mensaje "Incidencia registrada correctamente"
CN-05	Cancelación del conductor	Click en "Cancelar viaje"	Se descuentan 10 puntos y se reasigna otro conductor
CN-06	Llegar a 70 puntos	Repetir cancelaciones	El conductor queda suspendido temporalmente por 30 segundos
CN-07	Llegar a 0 puntos	Repetir cancelaciones	El conductor queda suspendido permanente
CN-08	Solicitar viaje con suspensión permanente	Solicitar un viaje	Aparece el mensaje "No hay conductores disponibles"

principales respondieron de acuerdo con el comportamiento esperado. El sistema permitió solicitar, aceptar y cancelar viajes correctamente, registró incidencias, aplicó sanciones al conductor cuando correspondía y ejecutó adecuadamente las suspensiones temporales y permanentes, así como la reasignación automática de conductores.

5. Conclusión

Las pruebas de Caja Negra permitieron comprobar que las funcionalidades implementadas cumplen con los requisitos establecidos. Los resultados obtenidos

fueron satisfactorios, ya que el sistema respondió correctamente ante todos los escenarios evaluados, garantizando un funcionamiento adecuado desde la perspectiva del usuario.

CAJA BLANCA

1. Objetivo

El objetivo de las pruebas de Caja Blanca es verificar el correcto funcionamiento de la lógica interna del sistema, analizando las estructuras de control, las condiciones, los caminos de ejecución y el flujo del programa. Estas pruebas permiten asegurar que cada decisión implementada en el código funcione de acuerdo con lo esperado.

2. Descripción de la prueba

Para realizar las pruebas de Caja Blanca se analizó el código fuente de la aplicación desarrollada en Python, identificando las funciones principales y las estructuras condicionales (if, elif, else) que controlan el comportamiento del sistema.

Se determinó que la función `cancelar_conductor()` contiene la mayor parte de la lógica del programa, ya que administra las sanciones aplicadas al conductor, la disminución del puntaje, la suspensión temporal, la suspensión permanente y la reasignación automática de conductores.

3. Funciones analizadas

Durante el análisis se identificaron las siguientes funciones principales:

4. Análisis de la función crítica

La función `cancelar_conductor()` fue seleccionada para el análisis debido a que concentra la mayor cantidad de decisiones del sistema.

Las principales condiciones evaluadas son las siguientes:

Funcion	Descripción
<code>solicitar_viaje()</code>	Asigna un conductor al usuario
<code>aceptar_viaje()</code>	Cancela el viaje solicitado
<code>cancelar_usuario()</code>	Cancelar Viaje (Usuario)
<code>reportar_incidencia()</code>	Registra una incidencia del usuario.
<code>cancelar_conductor()</code>	Aplica sanciones, suspensiones y reasigna conductores
<code>contar_suspension()</code>	Controla el tiempo de suspensión temporal
<code>actualizar()</code>	Actualiza toda la interfaz gráfica

Primera decisión

Condición:

Verifica si existe un conductor asignado al viaje.

Resultado esperado:

Si no existe un conductor asignado, la función finaliza sin realizar ninguna acción.

Si existe un conductor, continúa con el proceso de cancelación.

Segunda decisión

Condición:

Verifica si el puntaje del conductor es menor o igual a cero.

Resultado esperado:

El conductor recibe una suspensión permanente.

Se bloquea la posibilidad de aceptar nuevos viajes.

Tercera decisión

Condición:

Verifica si el puntaje del conductor es menor o igual a 70 puntos y aún no posee una suspensión temporal.

Resultado esperado:

El conductor es suspendido temporalmente durante 30 segundos.

Finalizado el tiempo de suspensión, vuelve automáticamente al estado "Activo".

Cuarta decisión

Condición:

Verifica si el índice del conductor supera el tamaño de la lista de conductores disponibles.

Resultado esperado:

Si se llega al último conductor de la lista, el sistema vuelve a asignar el primer conductor disponible.

En caso contrario, asigna el siguiente conductor de la lista.

5. Complejidad ciclomática

La función `cancelar_conductor()` presenta cuatro estructuras de decisión (if), por lo que la complejidad ciclomática se calcula mediante la siguiente expresión:

$$M = \text{Número de decisiones} + 1$$

$$M = 4 + 1 = 5$$

Esto indica que existen cinco caminos independientes que deben ser evaluados para garantizar una cobertura adecuada de la lógica implementada.

6. Caminos independientes

7. Casos de prueba de Caja Blanca disponible.

8. Conclusión

Las pruebas de Caja Blanca permitieron verificar el correcto funcionamiento de la lógica interna del sistema, comprobando que las estructuras de decisión implementadas responden de manera adecuada ante diferentes escenarios. Asimismo, el análisis confirmó que las funciones críticas ejecutan correctamente las acciones relacionadas con la asignación de conductores, la aplicación de sanciones, las suspensiones y la reasignación automática de viajes, garantizando el comportamiento esperado del sistema.

Camino	Descripción
P1	No existe un conductor asignado y la función termina inmediatamente
P2	El conductor cancela el viaje y el sistema aplica la sanción correspondiente y reasigna otro conductor
P3	El puntaje del conductor llega a 70 puntos y se aplica una suspensión temporal
P4	El puntaje del conductor llega a 0 puntos y se aplica una suspensión permanente
P5	El sistema alcanza el último conductor de la lista y vuelve automáticamente al primer conductor disponible

Camino	Condición evaluada	Resultado esperado
CB-01	No existe conductor asignado	La función finaliza sin realizar acciones
CB-02	El conductor cancela un viaje con puntaje mayora a 70	Se descuentan 10 puntos y se reasigna un nuevo conductor
CB-03	El puntaje del conductor llega a 70 puntos	El conductor queda suspendido temporalmente durante 30 segundos
CB-04	El puntaje del conductor llega a 0 puntos	El conductor recibe una suspensión permanente y ya no puede aceptar viajes
CB-05	Se alcanza el último conductor de la lista	El sistema vuelve a asignar automáticamente el primer conductor disponible

ANÁLISIS ESTÁTICO DEL CÓDIGO MEDIANTE SONARQUBE

1. Objetivo

El objetivo del análisis estático mediante SonarQube es evaluar la calidad del código fuente de la aplicación desarrollada en Python, identificando posibles errores, malas prácticas de programación, vulnerabilidades de seguridad y aspectos relacionados con la mantenibilidad del software.

2. Herramienta utilizada

Para realizar el análisis se utilizó SonarQube Community Edition, una herramienta de análisis estático que inspecciona el código fuente sin necesidad de ejecutar la aplicación. Esta herramienta permite evaluar diferentes métricas relacionadas con la calidad del software.

Entre los aspectos analizados se encuentran:

Detección de errores (Bugs).

Identificación de malas prácticas de programación (Code Smells).

Detección de vulnerabilidades de seguridad.

Evaluación de la mantenibilidad del código.

Complejidad del código.

Cantidad de líneas de código analizadas.

3. Procedimiento

Para realizar el análisis se siguieron los siguientes pasos:

Se instaló SonarQube Community Edition.

Se inició el servidor de SonarQube.

Se creó un nuevo proyecto para la aplicación desarrollada en Python.

Se configuró el archivo sonar-project.properties.

Se ejecutó SonarScanner sobre el proyecto.

Se visualizaron los resultados desde la interfaz web de SonarQube.

4. Resultados del análisis

Una vez finalizado el análisis, SonarQube genera un reporte con diferentes indicadores de calidad del código, entre ellos:

Número de Bugs encontrados.

Número de Code Smells.

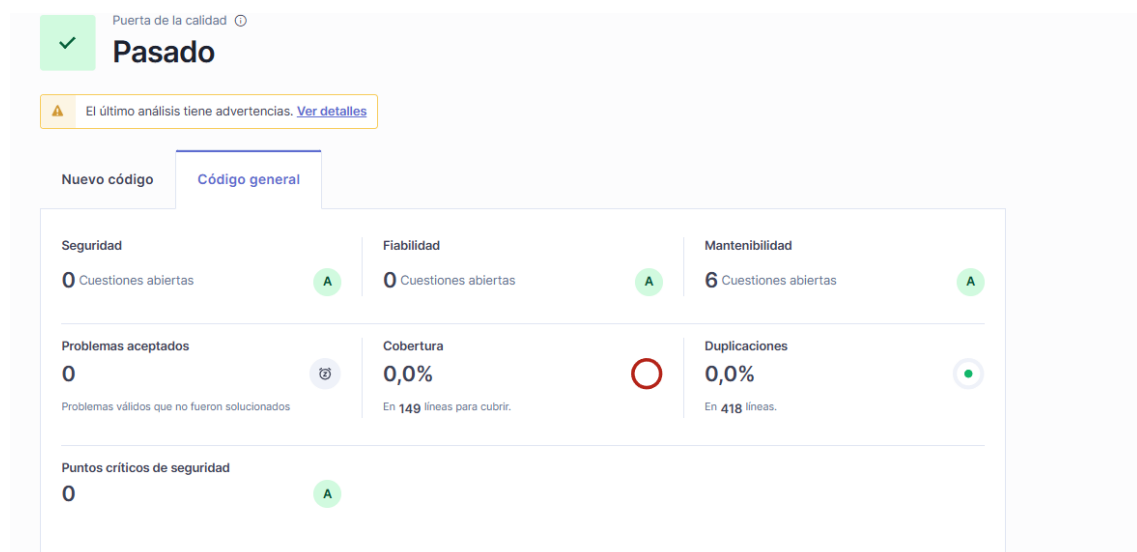
Vulnerabilidades de seguridad detectadas.

Complejidad del código.

Índice de mantenibilidad.

Líneas de código analizadas.

Descripción General



Problemas encontrados

☐	Defina una constante en lugar de duplicar este literal "SUSPENDIDO PERMANENTE" 3 veces.	Adaptabilidad
Mantenibilidad	Alto	diseño +
Abierto	No asignado	L72 + 6min de esfuerzo + Hacer 12 minutos

☐	Defina una constante en lugar de duplicar este literal "SUSPENDIDO TEMPORAL" 3 veces.	Adaptabilidad
Mantenibilidad	Alto	diseño +
Abierto	No asignado	L76 + 6min de esfuerzo + Hacer 12 minutos

☐	Fusione esta rama con la idéntica en la línea "73" o cambie una de las implementaciones.	Adaptabilidad
Mantenibilidad	Medio	diseño sospechoso +
Abierto	No asignado	L77 + Esfuerzo de 10 minutos + Hacer 12 minutos

☐	Agregue campos de reemplazo o use una cadena normal en lugar de una cadena f.	Intencionalidad
Mantenibilidad	Medio	confuso +
Abierto	No asignado	L129 + 1 minuto de esfuerzo + Hacer 12 minutos

☐	Agregue campos de reemplazo o use una cadena normal en lugar de una cadena f.	Intencionalidad
Mantenibilidad	Medio	confuso +
Abierto	No asignado	L199 + 1 minuto de esfuerzo + Hacer 12 minutos

☐	Agregue campos de reemplazo o use una cadena normal en lugar de una cadena f.	Intencionalidad
Mantenibilidad	Medio	confuso +
Abierto	No asignado	L265 + 1 minuto de esfuerzo + Hacer 12 minutos

Puntos críticos de seguridad

Puntos críticos de seguridad revisados

Para revisar

Reconocido

Fijo

Seguro



No hay puntos críticos de seguridad para revisar.

La próxima vez que analice un fragmento de código que contenga un riesgo potencial para la seguridad, aparecerá aquí.

[Obtenga más información sobre los puntos de acceso de seguridad](#)

5. Interpretación de resultados

El análisis realizado con SonarQube permitió evaluar la calidad del código fuente de la aplicación. Las observaciones obtenidas ayudan a identificar oportunidades de mejora relacionadas con la organización del código, la reducción de complejidad y la aplicación de buenas prácticas de programación.

Asimismo, esta herramienta facilita el mantenimiento del sistema al detectar problemas que podrían afectar su evolución o generar errores en futuras modificaciones.

6. Conclusión

El uso de SonarQube complementó las pruebas de Caja Blanca al realizar un análisis estático del código fuente. Gracias a esta herramienta fue posible evaluar la calidad del software desde el punto de vista técnico, identificando posibles mejoras y verificando que la estructura general del código cumple con criterios de mantenibilidad y buenas prácticas de desarrollo.

CONCLUSIONES

1. El análisis realizado permitió identificar deficiencias en el sistema actual de control de cancelaciones y penalizaciones de la plataforma Uber.
2. La propuesta desarrollada mejora el control de incidencias mediante la implementación de sanciones progresivas, historial de penalizaciones y disminución automática de puntos en la calificación del conductor.
3. El sistema de suspensión temporal automática contribuye a reducir cancelaciones repetitivas y fomenta un mejor comportamiento por parte de los conductores.
4. La integración del código de verificación incrementa la seguridad del servicio y garantiza una correcta validación entre pasajero y conductor antes de iniciar el viaje.
5. Los diagramas UML y prototipos desarrollados permitieron representar de manera clara el funcionamiento del sistema actual y del sistema propuesto.
6. El modelo incremental seleccionado facilita futuras ampliaciones y mejoras del sistema sin afectar completamente la operación de la plataforma.

RECOMENDACIONES

1. Implementar el sistema de penalizaciones progresivas dentro de la plataforma para reducir cancelaciones innecesarias y mejorar la calidad del servicio.
2. Realizar pruebas piloto con grupos reducidos de conductores antes de una implementación general.
3. Incorporar notificaciones automáticas para informar al conductor sobre sanciones, puntos descontados y tiempo restante de suspensión.
4. Mantener actualizado el historial de incidencias para garantizar transparencia y seguimiento adecuado del comportamiento de los conductores.
5. Aplicar revisiones periódicas del sistema para optimizar las reglas de penalización según el comportamiento observado en la plataforma.
6. Continuar fortaleciendo los mecanismos de seguridad y validación de identidad para mejorar la confianza tanto de pasajeros como de conductores.

7. Evaluar futuras integraciones con inteligencia artificial para detectar patrones de cancelación sospechosos y prevenir malas prácticas dentro del servicio.